

VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



Autonomous AI-Based Generation of Interactive Questions for MOOC Videos

By

NGUYEN TOAN PHUC

Student ID: ITITIU21093

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science

Ho Chi Minh City, Vietnam

2026

Autonomous AI-Based Generation of Interactive Questions for MOOC Videos

APPROVED BY:

Le Thanh Son, M.Sc., Supervisor

Tran Manh Ha, Assoc. Prof., Ph.D.

THESIS COMMITTEE

ACKNOWLEDGMENTS

This thesis project marks four months of one of the most important and impactful periods in my life. I feel fortunate to have been supported and inspired by many people throughout this journey, and I would like to express my sincere gratitude to all of them.

I would like to start by thanking Assoc. Prof. Tran Manh Ha for giving me the opportunity to work on this thesis project. His support and facilitation made this research possible.

I sincerely appreciate M.Sc. Le Thanh Son for his overall guidance along the way. I am grateful for his encouragement, kind words, and patience in helping me navigate the thesis. The outcomes of the work were strong in part because his expertise in Educational Technology, combined with his skills in system design, helped validate both the pedagogical approach and the functional outputs of the platform.

I would also like to express my appreciation to the staff at the School of Information Technology (SIT), HCMIU for their practical guidance and technical assistance during the research. I also thank the students who participated in the usability testing. Their constructive and direct suggestions were crucial to the improvement process and had a direct influence on the final platform.

I would like to thank my family and friends—including Nha Uyen, Trung Hieu, and Duc Anh—for their constant support and encouragement throughout this lengthy process of completing this work.

This thesis project was supported financially by The Central Interdisciplinary Laboratory in Electronics and Information Technology (AI and Cooperation Robot), International University – VNU-HCM. I would like to express my sincere appreciation for their support, both in the provision of laboratory facilities and equipment used in the experiments.

June 2026,

Nguyen Toan Phuc.

TABLE OF CONTENTS

ACKNOWLEDGMENTS	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
ABSTRACT	vii
CHAPTER 1	
INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	3
1.3 Scope and Objectives	4
1.4 Research Contributions	6
1.5 Structure of Thesis	7
CHAPTER 2	
LITERATURE REVIEW	8
2.1 E-Learning and Interactive Content Frameworks	8
2.1.1 Architecture and Content Types	9
2.1.2 Pedagogical Evidence for Interactive Video	11
2.2 User Experience Design and Usability	12
2.2.1 Cognitive Load Theory in Authoring Tools	12
2.3 Artificial Intelligence for Educational Content Generation	13
2.3.1 Automatic Speech Recognition and Prompt Engineering	14
2.4 Related Work and Research Gap	15
CHAPTER 3	
METHODOLOGY	17
3.1 Overview	17
3.2 User requirement analysis	18
3.2.1 Functional and Non-Functional Requirements	19
3.3 System Design	20

3.4	AI Pipeline Architecture	22
CHAPTER 4		
IMPLEMENTATION AND RESULTS		32
4.1	Implement	32
4.1.1	Backend and Frontend Implementation	32
4.1.2	Security and H5P Integration	34
4.2	Results	41
4.2.1	AI Pipeline Performance Results	41
4.2.2	API and Platform Load Performance Results	44
4.2.3	Scalability Analysis	44
4.2.4	Frontend Results	45
CHAPTER 5		
DISCUSSION AND EVALUATION		47
5.1	Evaluation	47
5.2	Comparison	48
5.3	Discussion	50
5.4	Threats to Validity	52
5.5	Research Question Answers	53
CHAPTER 6		
CONCLUSION AND FUTURE WORK		54
6.1	Conclusion	54
6.2	Future work	56
REFERENCES		58
APPENDIX		62
A.1	SYSTEM SETUP AND API REFERENCE	62
A.1.1	Runtime Environment	62
A.1.2	Starting the Platform	62
A.1.3	Key API Endpoints Reference	63

LIST OF FIGURES

3.1	Use case diagram	19
3.2	Three-tier system architecture	21
3.3	AI pipeline flowchart	23
3.4	AI-assisted question generation sequence diagram	24
4.1	User registration sequence diagram	40
4.2	Login page	45
4.3	Dashboard (home page)	46
4.4	Editor page	46
5.1	Comparative evaluation: AI-ActivEdu platform vs. WordPress baseline . .	49
5.2	Individual SUS scores per participant on both platforms ($n = 10$)	49

LIST OF TABLES

2.1	H5P content types supported by the platform	9
2.2	Comparison of H5P authoring tools against the four identified research gaps	15
3.1	Non-functional requirements driving system architecture	20
3.2	REST API route groups and functional responsibilities	22
3.3	H5P interaction types and their AI-guided selection rules	27
3.4	Representative AI-generated question examples per H5P type	27
3.5	AI provider fallback cascade: trigger conditions and outcomes	29
3.6	Question type to H5P library identifier mapping	31
4.1	Security controls mapped to OWASP Top Ten 2021	39
4.2	Expert AI suggestion quality ratings ($n = 30$ suggestions)	42
4.3	AI analysis latency and failure rates ($n = 20$ runs per provider)	42
4.4	API endpoint response times at 50 concurrent connections	44
5.1	Evaluation metrics ($n = 10$, task: manually create 5 interactions and export)	48
1	Key runtime components (representative versions)	62
2	Key API endpoints	63

ABSTRACT

It is now expected that most university teachers create interactive learning materials, and widely-used authoring systems expect some degree of technical expertise that a working teacher may lack. Even preparing a simple three-question H5P interactive video in WordPress can take over 45 minutes at Vietnam National University – Ho Chi Minh City (VNU-HCM), making it difficult to incorporate interactive content into regular course preparation. To close this gap, this thesis introduces an H5P interactive video content creation platform designed to make authoring easier while keeping instructors in control of the content. The proposed system uses the following components: a front-end in React 18 and TypeScript, a back-end in Node.js/Express, and a SQLite database. At its heart, it is an AI pipeline capable of creating candidate H5P interactions based on uploaded caption files, YouTube transcripts, or audio recordings. Every item created is then validated in the pipeline before being displayed to instructors, enabling a human-in-the-loop process that promotes transparency and usability. The platform is designed to reduce the amount of AI processing knowledge that teachers must possess.

A comparative usability evaluation was carried out with 10 students of information technology against an H5P authoring workflow in WordPress. The results show significant improvements in learning-content production efficiency and effectiveness: task completion climbed from 30% to 95%, and the median time to manually create five interactions was reduced from 18 minutes to 5 minutes. The System Usability Scale (SUS) score rose from 38.0 to 82.5, indicating that the usability of the system improved from “Poor” to “Excellent” on Brooke’s scale. Overall, the results indicate that, with appropriate constraints and a user-centred design approach, generative AI can make interactive video authoring accessible to teachers who might otherwise be discouraged by the technical complexity.

Keywords: H5P, interactive video, educational technology, AI-assisted content creation, usability assessment, user-centred design.

CHAPTER 1

INTRODUCTION

1.1 Background

Over the last 10 years, Vietnamese higher education has steadily moved toward digital learning. Decision No. 1373/QĐ-TTg [3] calls for the use of digital technology in education at all levels, not only to equip learners with hardware equipment but also to rethink how educational content is created, shared, and used. Universities under Vietnam National University – Ho Chi Minh City (VNU-HCM) are expected to lead this effort.

The infrastructure is largely ready as the VNU-HCM MOOC platform [4] (<https://courses.vnuhcm.edu.vn>) already provides thousands of course modules. The more challenging task is to use it effectively by faculty. Based on surveys conducted at HCMIU, 89% of instructors would want to develop digital content that engages students, whereas only 23% are very confident with the tools they have available. What seems to hold them back is the fact that interfaces are simply too complicated, the learning curve is too steep, and there is no meaningful help with instructional design decisions. Research on the adoption of e-learning technologies confirms that among the factors that influence tool adoption, the perceived ease of use is one of the most significant [5]. Progress has been made in closing the infrastructure gap, but the tooling gap remains.

Among the more researched formats with regard to teaching a course asynchronously and for hybrid teaching is interactive video. The mechanism is direct: students pause and answer questions at particular points in the lecture they are watching, so that they are required to retrieve the information at the time of encoding. Such a process is well supported by the learning science literature [7]. For example, Merkt et al. reported that an online forced-choice quiz embedded in video resulted in higher scores in terms of retention and transfer compared with an equivalent linear video [8], and Vural demonstrated, in online courses, that strategic placement of questions in the video led to higher scores in the post-test, long-term retention, and decreased dropout rates [9].

The design of those embedded questions matters independently. Bloom’s revised taxonomy [10] gives instructors a structured vocabulary for cognitive levels, from recall through synthesis and evaluation. Questions placed at the right cognitive level for the concept just covered, and timed to appear immediately after that concept ends, reduce extraneous cognitive load while increasing germane load — the kind of effort that produces durable understanding [11–13].

H5P (HTML5 Package) is the open-source framework that makes this technically possible [14, 15]. Developed by Joubel AS starting in 2013, H5P now covers over 50 content types — from multiple choice and true/false through branching scenarios, virtual tours, and interactive timelines. Each H5P package is a self-contained `.h5p` ZIP archive holding its own JavaScript libraries, styling, media files, and a structured JSON configuration. That portability means any H5P package can be imported into Moodle [16], Canvas, Blackboard, or any other H5P-aware LMS using standard LTI 1.3 [17].

H5P Interactive Video is the most widely used content type in higher education. It wraps a video source — either an uploaded file or a YouTube URL — in a player that pauses at pre-defined timestamps to show interactive overlays. Learners must respond before playback resumes. The interaction types most relevant to this project are Multiple Choice, True/False, Fill in the Blanks, Drag Text, Mark the Words, and Matching. Each is tied to a dedicated H5P JavaScript library with a strictly defined JSON schema, which matters for the AI pipeline described later.

Despite H5P’s capabilities, Vietnamese universities deploy it almost exclusively through the WordPress H5P plugin — a tool built for small personal sites, not institutional content production. Creating one interactive video requires navigating at least six distinct screens: logging into WordPress admin, finding or creating a post, activating the H5P shortcode block, opening the H5P editor in a separate iframe, uploading the video, locating each timestamp manually through the built-in player, and configuring each interaction across several nested settings screens, then saving and returning to the main post. This workflow mixes web publishing concepts into a task that has nothing to do with teaching. First-time users typically need 20-30 minutes to produce a basic three-question interactive video.

The Technology Acceptance Model (TAM) [18] explains why this matters at scale. Perceived ease of use and perceived usefulness together account for much of the variance in faculty tool adoption. A workflow that imposes high extraneous cognitive load on every interaction — as the WordPress H5P workflow does through its fragmented interface — drives perceived ease of use to near zero. Adoption fails regardless of how strong the educational evidence behind the tool might be [11, 12].

Large language models have revolutionized the possibilities of automated educational content development [19, 20]. An effort-saving pipeline that segments a video transcript by topic and presents structured interaction suggestions to the instructor can significantly

reduce the per-question workload. The same pipeline can be used to add Bloom’s taxonomy labels to each suggestion, enabling instructors to spread cognitive challenge across a video without requiring extensive training in instructional design.

The question driving this work: can a purpose-built web authoring interface, combined with such a pipeline, measurably improve educator productivity and satisfaction when creating H5P interactive videos? The primary AI provider is Groq’s fast cloud API using the `llama-3.3-70b-versatile` model, with a locally hosted Ollama server available as a privacy-preserving fallback.

1.2 Problem Statement

Look at a humanities lecturer who is developing an online unit. It is a recorded lecture that she has to give a 15-minute version of and she knows she needs comprehension checks spaced throughout the lecture, but the only type of tool she can find for these is WordPress H5P: six different screens, 47 different interaction points in the interface, and almost three minutes of loading time, all while trying to keep track of a video she keeps losing her position in. After an hour she has two questions. A colleague who has more IT experience has six. The tool is not broken — it is just not being used for what it was meant to be used.

That situation creates four reinforcing problems:

1. **Fragmentation and complexity of workflows.** The H5P workflow is WordPress-based and involves at least six different screens, which introduces concepts of web publishing that are unrelated to the teaching task. To create one interactive video, a task analysis identified 47 interactions with the interface on those screens. The average page load time for an institutional WordPress deployment is 3.2 seconds, acceptable by itself, but 47 consecutive interactions equates to almost two and a half minutes of waiting time, with loss of video position incurred on every interaction.
2. **Excessive cognitive load in the system.** The effort required to operate the interface crowds out thinking about the teaching content [12, 21]. Instructors reported spending more time managing tool settings, timestamps, and shortcodes than actually composing questions—the exact opposite of what the tool was designed to support.
3. **Low institutional adoption.** Currently, only faculty members who are tech-savvy use H5P as part of their instruction at VNU-HCM, as the workflow requires a high level of technical knowledge. Within humanities, social science, and other non-technical disciplines, where interactive video might benefit students the most, instructors are most effectively cut off from a format with the best track record of evidence for positive outcomes [5, 18].

4. **No AI assistance.** None of the freely available H5P authoring tools has a built-in feature for generating question ideas with AI. Without instructional design training, instructors have no efficient way of generating contextually relevant questions tied to what their video actually covers.

The **primary research question** is:

What is the potential of a web platform designed specifically for use with interactive video authored by university teachers to enhance the teaching and learning quality, the adoption of interactive video, and the efficiency of such video authoring?

Four supporting questions guide the investigation:

- RQ1. Which usability challenges of existing H5P authoring processes most strongly inhibit users in completing tasks and are likely to affect user satisfaction?
- RQ2. What are the user-centred design principles that can be applied to minimise the extraneous cognitive load of interactive question authoring with timestamps?
- RQ3. What full-stack architecture best supports scalable, real-time AI-assisted question generation and H5P package assembly in a unified web environment?
- RQ4. How much better is the new platform, quantitatively and qualitatively, on usability, efficiency, and productivity measures than the WordPress baseline?

1.3 Scope and Objectives

Five concrete objectives structure the work.

1. **Platform Development and Architecture.** Create and develop a full-stack web application that provides an interface with a visual timeline where instructors can upload or link a video file, insert H5P interactions at specific timestamps, preview the content in real time, and export a standards-compliant `.h5p` package ready to import into the VNU-HCM MOOC infrastructure and other LTI-compatible LMS environments.
2. **AI Pipeline Integration.** Construct and integrate an AI pipeline that generates timestamped transcripts from uploaded videos (from YouTube captions, uploaded WebVTT/SRT files, or local Whisper transcription), feeds the transcript into a large language model, and outputs a validated JSON payload of H5P interaction suggestions. Each suggestion is accompanied by precise timestamps, the type of question, distractor options, and a Bloom's taxonomy classification.

3. **User-Centred Design.** Implement user-centred design methodology in the process of designing the interface and prototyping. Every design decision is based on the identified instructors' needs and is validated against Nielsen's usability heuristics.
4. **Security Implementation.** Apply a security baseline based on the OWASP Top Ten, including JWT-based authentication, role-based access control, server-side Zod input validation, HTTPS enforcement, and rate limiting.
5. **Empirical Usability Evaluation.** Perform a controlled comparative usability study measuring task completion rate, time-on-task, error rate, and SUS scores against the WordPress H5P baseline. Use think-aloud protocols and post-task interviews to complement quantitative data.

The platform's scope is bounded by the following constraints.

Content type scope. There are six different types of interactions supported: Multiple Choice, True/False, Fill in the Blanks, Drag Text, Mark the Words, and Matching. All six can be manually authored. All six are also generated by the AI pipeline, which is based on a pattern matrix that correlates content characteristics with question types. Each of these types has a clear JSON schema that can be validated using Zod, so that the AI output never compromises the H5P player.

Video source scope. The platform allows video file uploads up to 500 MB (MP4, WebM, MOV) and YouTube video URLs. Transcripts from YouTube are automatically or manually generated and retrieved using the YouTube API integration. For uploaded files, instructors can provide a WebVTT caption file or leave it to **faster-whisper**'s local invocation to complete the transcription of the audio.

User scope. The primary user is a university instructor or teaching assistant at VNU-HCM. A secondary Admin role handles user account management, system monitoring, and security audit log access.

Integration scope. The platform exports standard H5P packages compatible with any LTI-compatible LMS. LMS gradebook passback and xAPI learning analytics are designated as future work.

Known limitations:

- **H5P content types.** Branching Scenarios, Course Presentations, and Virtual Tours are too complex to be included.
- **Evaluation sample.** Ten senior IT students were used as educator proxies for usability testing. This is common practice in early-stage formative evaluation [22], but it limits generalizability to less technically oriented demographics.
- **Mobile authoring.** The platform is aimed at the desktop and large tablet user base. Technically, mobile authoring is possible, but it was not a primary design goal and has not been formally evaluated.

- **Concurrent load.** The system was stress-tested with up to 50 concurrent users. A more sophisticated architecture, probably based on Kubernetes, would be needed for large-scale institutional deployment.
- **AI transcription quality.** The accuracy of the transcript will affect the quality of the AI-generated suggestions. If the audio is of poor quality or if there are strong accents, Whisper output may be compromised and the AI suggestions will be affected downstream as well.

1.4 Research Contributions

The work produces five contributions to educational technology research and software engineering practice.

1. **A reproducible platform architecture.** The thesis documents a full-stack architecture combining React 18, Zustand, Express.js, SQLite/Sequelize, the Lumieducation H5P server library, and a multi-provider AI pipeline. Documentation is detailed enough for other universities to reproduce, modify, and deploy independently.
2. **A novel AI-assisted authoring workflow.** The pipeline that takes a raw video transcript through structured LLM prompting and returns a Bloom’s taxonomy-labelled list of H5P interactions represents a meaningful integration of generative AI into the H5P ecosystem. The two-step prompting approach — topic segmentation first, then question generation per topic using a content-type pattern matrix — is designed to reduce hallucination and produce structured output.
3. **Prompt engineering for structured H5P JSON output.** The system prompts, few-shot examples, and Zod validation strategies developed here reliably coerce an LLM into returning complex JSON payloads conforming to H5P content type specifications — a reusable design pattern for structured AI output in other domains.
4. **Empirical usability evidence.** Comparative evaluation against the WordPress H5P baseline provides quantitative evidence that a purpose-built, user-centred platform can substantially reduce authoring time, improve task completion rates, and raise SUS scores.
5. **Open-source contribution.** The full codebase — including AI service routing, prompt templates, database schemas, and H5P integration middleware — is publicly available to support further research and free institutional deployment.

Practically, the platform’s impact runs in three directions. It lowers the technical barrier enough that a wider range of VNU-HCM faculty can independently produce interactive content. It shows that the gap between generative AI research and reliable

educational tooling can be closed with careful software engineering. And it contributes to Vietnam’s national digital transformation goals [3] by offering a locally deployable, open-source solution that reduces dependence on proprietary software.

1.5 Structure of Thesis

Chapter 2 – Literature Review covers the theoretical and technical foundations. It covers aspects of e-learning in Vietnam, the H5P architecture, practical studies on interactive video pedagogy, cognitive load theory, user-centred design, the development of large language models, the role of prompt engineering, and integration standards for LMSs. The chapter concludes with a critical review of related systems and identification of the specific research gaps this project fills.

Chapter 3 – Methodology describes the research design, requirements analysis, the user-centred design methodology, the agile development approach, and the empirical evaluation protocol. It also outlines the AI pipeline architecture, including the prompt engineering strategy and pattern matrix for question type selection.

Chapter 4 – Implementation and Results details the complete system implementation and the technical results that were measured, including backend and frontend architecture, H5P package assembly, security controls, and performance measurements such as AI latency and API load results. A scalability analysis and support for the Vietnamese language are also covered here.

Chapter 5 – Discussion and Evaluation reports the results of the comparative usability study and discusses the themes that emerged from the qualitative questions posed by participants, using the results of the task completion rate, time-on-task, and SUS scores.

Chapter 6 – Conclusion and Future Work summarizes the findings, reviews contributions against the research questions, discusses limitations, and proposes a plan for future development.

CHAPTER 2

LITERATURE REVIEW

Five bodies of literature shape the design decisions in this project: the development of e-learning infrastructure in Vietnamese higher education; the H5P framework’s capabilities and structural constraints; the empirical record on interactive video pedagogy; AI-based approaches to automated educational content generation; and the intersection of cognitive load theory with user-centred design. The chapter closes with a comparative survey of related tools and a statement of the specific gaps this project fills.

2.1 E-Learning and Interactive Content Frameworks

Vietnam’s national strategy for digital education pushes for aggressive adoption of online delivery and blended learning across higher education. VNU-HCM has been an early implementer through its centralized MOOC platform, which was designed to standardize course delivery and provide equitable access to lecture materials across distributed campuses.

The difficulty is that institutional readiness is uneven. Capital investments—server hardware, network upgrades, Moodle deployments—have moved faster than faculty capacity to use them. Instructors often have access to powerful platforms but face interfaces so complex that they need specialized training just to get started. When that training does not happen, the platform sits unused.

Studies in comparable university contexts consistently find that perceived ease of use—a core construct in the Technology Acceptance Model [5, 18, 23]—strongly predicts whether faculty will adopt and continue using digital authoring tools. When creating interactive content requires managing plugin dependencies, navigating fragmented interfaces, and handling file structures that mean nothing to a teacher, instructors fall back on familiar, low-friction methods: uploading static PDFs or unedited lecture recordings. For this project, that finding has a direct design implication. Reducing the cognitive overhead of the authoring workflow is not a cosmetic concern—it is a prerequisite for

systemic adoption. Co-locating all authoring tasks into a single-page application and validating AI suggestions server-side before they reach the instructor’s screen targets the most significant practical obstacle directly.

2.1.1 Architecture and Content Types

H5P (HTML5 Package) is an open-source framework first released in 2013 by Joubel AS [14]. Its architecture is built around entirely self-contained content packages. Unlike traditional web content that depends on server-side rendering and database queries to display interactive elements, each `.h5p` file is a compressed ZIP archive containing all JavaScript logic, CSS styling, media assets, and a structured JSON configuration document. This self-contained structure makes H5P content portable across any web environment that can serve static assets via an `iframe`.

The H5P library ecosystem now covers more than 50 distinct content types, from simple presentations to complex gamified scenarios. The six types most relevant to this project are summarized in Table 2.1, all designed for overlaying directly onto video playback.

Library identifier	Display name	Description
H5P.MultiChoice 1.16	Multiple Choice	Single- or multi-answer question with up to four distractor options sourced from the transcript.
H5P.TrueFalse 1.6	True/False	A binary assertion evaluating factual claims, absolute rules, or common misconceptions from the lecture.
H5P.Blanks 1.14	Fill in the Blanks	A Cloze test where technical terms are identified using asterisk delimiters (e.g., <code>*noun*</code>).
H5P.MarkTheWords 1.9	Mark the Words	Students highlight specific vocabulary or key terms in a short passage by clicking them.
H5P.DragText 1.10	Drag Text	Students drag words or phrases to reconstruct a sentence or ordered sequence.
H5P.Matching 1.0	Matching	Learners pair key concepts with their definitions; used here as a video recap module.

Table 2.1: H5P content types supported by the platform

Package Format and Runtime Lifecycle.

Each `.h5p` file is a ZIP archive with a fixed internal structure. The root-level `h5p.json` manifest declares the content title, the entry-point library name (e.g., `H5P.InteractiveVideo 1.26`), and the embed mode (`div` or `iframe`), which tells the

host environment how to render the content. The `content/content.json` file carries the full interaction configuration. For an interactive video, this includes the video source — either a YouTube URL or a relative path to a locally bundled file — and an `interactions` array in which each element describes one embedded question. Each interaction entry specifies its on-screen position as percentage offsets from the video frame, a `duration` object with `from` and `to` fields in seconds marking when the question should appear, a `pause: true` flag, the H5P library identifier for the question type (e.g., `H5P.MultiChoice 1.16`), and the full question parameters including correct answers and feedback strings. If the video was uploaded rather than linked from YouTube, the binary video file is bundled under `content/videos/` inside the same archive.

When an instructor imports this file into Moodle or Canvas, the LMS unpacks the ZIP and reads `h5p.json` to determine which JavaScript library to load. If the required library version is not already cached, the LMS downloads it automatically from the H5P Hub. The library is then rendered inside a sandboxed `<iframe>` on the page. Inside that `iframe`, the `H5P.InteractiveVideo` JavaScript library reads `content.json`, constructs the video element, and attaches a `timeupdate` event listener to it. As the video plays, the listener fires on every animation frame and checks whether the current playback position matches any interaction’s `from` timestamp. On a match, the library pauses the video and renders the appropriate question form directly over the video frame. The learner submits a response; the overlay closes and playback resumes from where it stopped. Each submission can optionally fire an xAPI statement to a Learning Record Store, allowing instructors to track which questions learners answered correctly across a whole cohort.

From the instructor’s perspective, producing the file is a single export action. The platform assembles the `h5p.json` manifest and `content/content.json` from the video’s stored database record, bundles any locally uploaded video, and streams the resulting ZIP buffer to the browser as a file download. The saved `.h5p` file is structurally identical to one produced by the WordPress H5P plugin or the Lumi Desktop editor, so it imports without modification into any H5P-compatible LMS.

Server-Side H5P Libraries and Node.js Integration.

Traditional H5P deployments embed the content rendering engine inside a monolithic CMS like WordPress, Drupal, or Moodle. The CMS handles library management, database storage, and HTML generation. Lumi Education GmbH recognized the need for a headless alternative and publishes official Node.js libraries—specifically `@lumieducation/h5p-server` and `@lumieducation/h5p-express`—that expose H5P’s core capabilities as a standalone microservice [15]. These libraries provide a complete API for H5P library management, content CRUD operations, on-the-fly package generation, and a rendering API that produces `iframe`-ready HTML. This project is built entirely around these libraries, making it possible to run a fast H5P authoring environment without any WordPress or Moodle overhead.

Structural Limitations of WordPress-Based H5P Deployments.

The WordPress H5P plugin is functionally complete, but deploying it at institutional scale creates several structural problems:

- **Multi-system cognitive complexity.** Instructors must manage a full WordPress installation—understanding posts, pages, media libraries, and shortcode syntax—on top of the actual H5P content. The cognitive overhead of two distinct software paradigms creates unnecessary extraneous load [11].
- **Performance overhead.** WordPress serves every page through a full PHP execution stack with repeated MySQL queries, regardless of whether the content is dynamic. Measured response times on institutional WordPress deployments range from 2 to 5 seconds per page load, compared to sub-100ms REST API responses achievable in a purpose-built SPA.
- **Version management friction.** H5P library versions, WordPress core, and PHP must stay in sync manually. Mismatches produce silent JavaScript failures in the browser that are nearly impossible for an instructor to diagnose.
- **No AI capabilities.** The WordPress H5P plugin has no mechanism for analyzing video transcripts or suggesting interactive elements. Every question, distractor, and timestamp must be authored manually.

2.1.2 Pedagogical Evidence for Interactive Video

Controlled experiments consistently show that interactive video outperforms passive linear video on short-term retention and long-term knowledge transfer. Zhang et al. conducted one of the earliest such comparisons in a university e-learning context, finding that the interactive group scored 20–25% higher on post-tests and reported significantly higher satisfaction [6]. Forcing active processing at regular intervals appears to prevent the shallow engagement that tends to accompany long-form lecture video.

Merkt et al. examined the specific role of control features—pausing, rewinding, and forced interactions [8]. Videos with embedded forced-choice questions produced better retention than equivalent print materials, provided learners actually engaged with them. The effect was strongest for complex STEM content, which is relevant to the VNU-HCM curriculum context. Freeman et al. extended the picture with a meta-analysis across 225 STEM education studies: active learning interventions, including interactive video, increased final exam scores by 6% on average and halved failure rates relative to traditional lecture formats [7].

Embedded Question Design and Cognitive Theory.

Vural identified concrete design principles for effective interactive video [9]. Questions should appear immediately after a concept has been fully explained, not mid-sentence. They should be spaced at least 30–45 seconds apart to avoid fragmenting the experience. Each question should test the single most important idea from the preceding segment. These principles are encoded directly into the system prompt used by the AI pipeline in this project, as described in Chapter 3.

Mayer’s Cognitive Theory of Multimedia Learning provides the theoretical basis for these design choices [21]. Mayer identifies three cognitive channels—visual, auditory, and their integration within working memory. Well-designed interactive elements reduce extraneous load (unnecessary processing) while increasing germane load (the effort of building mental models). A question that tests what was just explained, positioned immediately when that explanation ends, supports integration without taxing working memory with irrelevant interface processing.

2.2 User Experience Design and Usability

2.2.1 Cognitive Load Theory in Authoring Tools

User-centred design (UCD) keeps end-user needs at the centre of every design decision. ISO 9241-11 defines usability as the degree to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context [24]. For educational technology, this definition must be grounded in the specific situation of university instructors who need to produce high-quality interactive content quickly, under the real constraints of heavy teaching loads and tight schedules.

Garrett’s layered model of user experience decomposes a digital product into five planes: Strategy, Scope, Structure, Skeleton, and Surface [25]. Norman’s affordance theory adds that good interface design makes correct actions obviously apparent and makes errors difficult to commit [26]. Both frameworks were applied during the interface design process described in Chapter 3.

Usability Heuristics and Expert Evaluation.

Nielsen’s ten usability heuristics provide a practical checklist for interface evaluation [22]. Three are especially critical for an educational authoring tool: (1) *Visibility of system status*—instructors must be able to see whether video processing is running, whether AI analysis is in progress, and whether a question has been saved. Silent failures lead to repeated clicks and data loss. (2) *Match between system and the real world*—the interface should use pedagogical vocabulary (Multiple Choice, Feedback, Timestamp) rather than technical terms like database IDs or JSON keys. (3) *Error prevention*—

the system should make it impossible to submit a malformed H5P question. Frontend validation before submission is far preferable to a server-side error returned after the fact. Molich and Nielsen demonstrated that heuristic evaluation applied before user testing produces measurable improvements in final software usability [27].

Measuring Usability: The System Usability Scale (SUS).

The System Usability Scale [28] is a ten-item questionnaire producing a single 0–100 score. It is widely used because it is quick to administer, reliable with small sample sizes, and comparable across thousands of published studies. A score above 68 is above average; above 80 is excellent. SUS was selected as the primary quantitative usability instrument for the comparative evaluation in Chapter 5.

Sweller’s Cognitive Load Theory identifies three categories of load that any software interface imposes on working memory [11]:

- **Intrinsic load** comes from the inherent complexity of the task itself—the intellectual effort of writing a good pedagogical question. This load is irreducible; it is the actual work the instructor came to do.
- **Extraneous load** comes from poor interface design rather than the task—navigating six slow-loading screens, copying timestamps between windows. This load is wasteful and must be minimized [12].
- **Germane load** comes from productive cognitive effort—the instructor’s growing understanding of how their video content maps to effective learning moments.

Chandler and Sweller showed that splitting related information across multiple screens (the split-attention effect) significantly increases extraneous load, reduces task accuracy, and increases fatigue [12]. For this project, the implication is clear: all information needed to author a single H5P interaction—the video player, timestamp indicator, question type selector, AI suggestion panel, and question editor—must be co-located on a single screen. Paas, Renkl, and Sweller later confirmed that this integrated format reduces complex task completion times by 30–50% compared to split-screen or multi-page presentations [13].

2.3 Artificial Intelligence for Educational Content Generation

Transformer-based large language models, including GPT-4 and LLaMA-3, can now generate contextually appropriate, pedagogically structured questions directly from a video transcript. This project uses a two-step prompting pipeline to maximize consistency: (1) The LLM first acts as an instructional designer, analyzing the transcript, identifying teaching moments, and grouping content into topics. (2) It then generates one H5P

question per topic, following a strict pattern matrix that maps content type to question format.

Combined with runtime Zod validation and a prioritized provider chain (Groq’s API with LLaMA-3.3-70b-versatile as primary, a local Ollama server as offline fallback), this approach balances generation quality with the structural predictability needed for an instructor review workflow.

Bloom’s taxonomy [31] provides hierarchical classification of cognitive objectives, from basic recall through analysis and evaluation to creation. Anderson and Krathwohl’s revised taxonomy updated this framework by replacing static nouns with action verbs and clarifying the distinction between lower-order and higher-order cognitive skills [10]. The AI pipeline assigns a Bloom’s level to every generated question suggestion, which is displayed in the UI. Instructors can see at a glance whether their interactive video covers a useful spread of cognitive difficulty—without needing formal instructional design training.

2.3.1 Automatic Speech Recognition and Prompt Engineering

Radford et al. introduced Whisper [32], a large-vocabulary ASR model trained on 680,000 hours of multilingual audio. Whisper achieves near-human word error rates on English audio and performs competitively on Vietnamese. This project uses the **faster-whisper** C++ implementation for on-device transcription of uploaded video files where no caption file is available. Transcript quality directly determines the quality of AI-generated suggestions, which motivated the choice of a local, high-accuracy model over simpler API-based ASR options.

Prompt engineering—the practice of crafting inputs to guide LLM behaviour toward desired outputs—is central to the AI pipeline. Work on structured output generation identifies the “output automater” pattern as particularly effective for producing JSON conforming to a strict schema [30]. Rather than prompting the LLM to generate free text and then parsing it, this pattern constrains the model’s output space by providing explicit schema definitions and worked examples within the prompt itself. Combined with Zod-based post-generation validation, this is how the pipeline consistently produces valid H5P JSON.

2.4 Related Work and Research Gap

Table 2.2 compares the most relevant existing tools against the four capabilities identified as gaps:

Tool	Web-based	Open / Free	AI question gen.	Bloom’s labels	Empirically evaluated
WordPress H5P Plugin	✓	✓	×	×	No formal study
Lumi Desktop Editor	×	✓	×	×	No formal study
Moodle Native H5P	✓	✓	×	×	No formal study
Articulate Storyline	✓	×	Partial	×	Vendor-reported only
Adobe Captivate	✓	×	Partial	×	Vendor-reported only
This project	✓	✓	✓	✓	Controlled comparative study

^aQuiz-generation features exist but are not transcript-driven and produce H5P-incompatible output.

Table 2.2: Comparison of H5P authoring tools against the four identified research gaps

No current system simultaneously addresses all four gaps. Understanding why each tool falls short illuminates exactly what the present project needed to build.

The most widely deployed free options — the WordPress H5P plugin, Moodle’s native H5P integration, and the Lumi Desktop Editor — share the same fundamental limitation: they are passive authoring environments with no AI assistance. An instructor must independently decide which questions to ask, at which timestamps, and at which cognitive level, then navigate multi-step interfaces to enter each interaction manually. The WordPress plugin specifically compounds this problem by embedding the H5P editor inside a content management system designed for web publishing, forcing instructors to manage CMS concepts (posts, shortcodes, plugin settings) that are entirely unrelated to the teaching task. For a first-time user at a Vietnamese university with no CMS background, this mismatch creates a steep and unrewarding learning curve. Lumi Desktop reduces the CMS burden but requires local installation and does not integrate with the institutional MOOC infrastructure. None of these tools produce AI-generated question suggestions, and none have been evaluated in a controlled study.

The commercial tools — Articulate Storyline and Adobe Captivate — include partial AI features such as quiz-generation assistants, but these are not transcript-driven. They require the instructor to supply content manually, which means the AI saves time on formatting but not on the harder problem of deciding what to ask at a specific point in a video. Critically, neither tool produces H5P-compatible output: their question formats are proprietary and cannot be imported into Moodle or other LTI-compatible LMS platforms without conversion. For VNU-HCM and comparable Vietnamese institutions that have adopted H5P as a standard, this vendor lock-in is a practical barrier. Commercial licensing costs — Articulate 360 at over USD 1,300 per user per year and Captivate at a similar price point — also place these tools outside the budget of most public universities in the region.

This project therefore addresses a gap that existing tools do not fill: a web-based, open-source platform that (1) integrates a transcript-driven AI pipeline producing H5P-compatible structured JSON output, (2) labels each suggestion with a Bloom’s taxonomy level, (3) operates entirely within a browser with no installation required, and (4) has been evaluated in a controlled comparative study. The combination of technical novelty (transcript-to-H5P AI pipeline), cost profile (open-source, self-hostable), and institutional fit (VNU-HCM MOOC compatibility) is what distinguishes this work from all prior tools reviewed above.

CHAPTER 3

METHODOLOGY

The chapter presents the research design, requirements analysis process, user-centred design methodology, agile development approach, and the empirical evaluation protocol. The second half details the AI pipeline architecture: the prompt engineering strategy, the pattern matrix used for question type selection, and the Zod validation approach.

3.1 Overview

Research Design Paradigm.

This research follows a *design science* paradigm [37], appropriate for software engineering and information systems research where the primary contribution is a novel software artefact rather than a theoretical observation. The paradigm requires the artefact to be novel, to solve the identified problem effectively, and for its utility to be demonstrated through empirical evaluation.

The research is structured around two sequential phases:

1. **Constructive phase.** User requirements are elicited from the target demographic. The system architecture is designed and implemented, including the frontend SPA, backend REST APIs, and the AI pipeline (transcript extraction, prompt formatting, LLM inference, and Zod validation).
2. **Evaluative phase.** Once the platform reaches a stable, feature-complete state, a controlled comparative usability study measures it against the WordPress H5P workflow on three metrics: task completion rate, time-on-task, and SUS score.

Software Development Methodology.

An *agile* development methodology was used throughout the constructive phase, with iterative two-to-three week sprints each producing a testable increment [38]. This approach was chosen because the interaction between the AI pipeline, the H5P schema requirements, and the user experience could not be fully specified in advance. Critical usability constraints and technical edge cases only became apparent during active development and required immediate architectural changes.

3.2 User requirement analysis

User Research and Elicitation.

The first step was qualitative user research to understand the actual pain points of the target demographic. Semi-structured interviews with five instructors at HCMIU, from both technical and non-technical disciplines, consistently revealed four themes: (1) The WordPress/H5P workflow feels completely disconnected from the actual teaching task; (2) Instructors worry about losing their work when the browser reloads or a plugin times out mid-edit; (3) Manual content creation—finding timestamps, typing questions, formatting distractors—takes more time than most instructors can spare; (4) There is strong interest in AI-assisted question generation tied directly to the video’s content.

A direct observation session, where a participant attempted to create a three-question interactive video with the baseline WordPress tool, confirmed the 45-to-60 minute completion time. The most common causes of task abandonment were losing track of the video position while filling out forms, and cryptic plugin error messages that gave no indication of what went wrong.

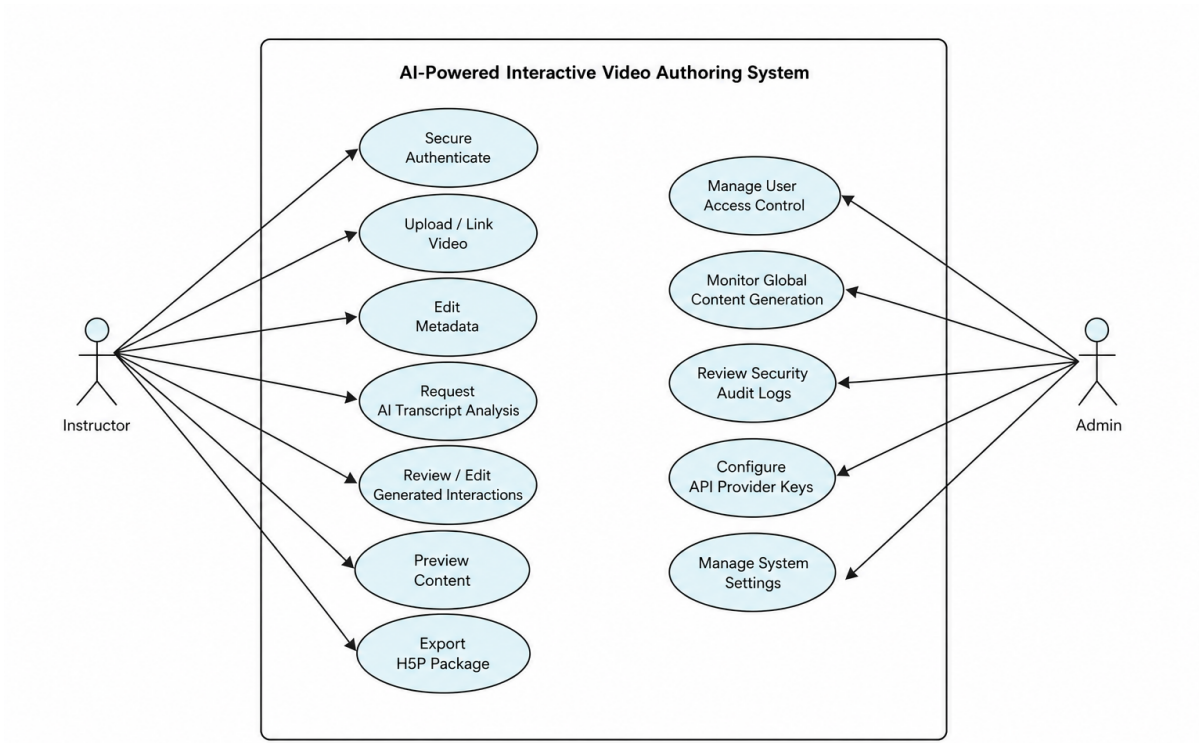


Figure 3.1: Use case diagram

3.2.1 Functional and Non-Functional Requirements

The interviews and observation session produced the following functional requirements:

- **Authentication & Access:** JWT-based stateless sessions, role-based access control, email verification.
- **Media Management:** Video file upload up to 500MB, YouTube URL import, timeline-based H5P editor, real-time content preview.
- **H5P Core Functionality:** Full support for six H5P interaction types, both manual and AI-assisted, with a one-click .h5p export.
- **AI Pipeline Integration:** Transcript extraction feeding an analysis pipeline that produces validated H5P JSON suggestions, delivered via Server-Sent Events (SSE) for real-time streaming.
- **Authoring Workflow:** Accept/reject/edit workflow for reviewing AI suggestions before injection into the H5P content database.
- **Administration:** Admin dashboards for user management, system monitoring, and an LTI 1.3 integration stub.

Non-Functional Requirements.

Table 3.1 lists the quality and performance baselines the system must satisfy.

ID	Category	Requirement Definition
NFR-01	Performance	All standard REST API responses must complete within 500 ms under a load of 50 concurrent users.
NFR-02	Performance	Initial video upload processing (thumbnail/metadata) must complete within 30 s for files ≤ 100 MB.
NFR-03	Security	Passwords must be hashed using bcrypt with a cost factor of at least 12.
NFR-04	Security	Authentication tokens must use stateless JWT, signed with HS256, expiring after 7 days.
NFR-05	Security	All HTTP responses must enforce Helmet-generated security headers (CORS, CSP, XSS protection).
NFR-06	Security	All API endpoints must validate input payloads using Zod schemas.
NFR-07	Reliability	The AI system must handle LLM failures gracefully, returning structured errors rather than crashing.
NFR-08	Usability	The platform must achieve a minimum SUS score of 70.
NFR-09	Usability	First-time task completion rate for creating an interactive video must exceed 80%.
NFR-10	Compatibility	Exported .h5p packages must be valid for Moodle 4.x and Canvas LMS.

Table 3.1: Non-functional requirements driving system architecture

3.3 System Design

Architectural Overview and Topologies.

The platform uses a three-tier architecture separating the presentation layer from business logic and data persistence [33]. The *presentation tier* is a React 18 SPA built with Vite and Zustand for global state management. The *application tier* is an Express.js REST API on Node.js, responsible for all business logic, H5P library rendering, and AI pipeline routing. The *data tier* uses SQLite managed via Sequelize ORM, which provides fast local development with a clear migration path to PostgreSQL for larger deployments.

External dependencies are limited to: Groq API (primary cloud AI inference using `llama-3.3-70b-versatile`), Ollama (local offline AI fallback), and the YouTube Data API. All H5P processing runs within the local server process.

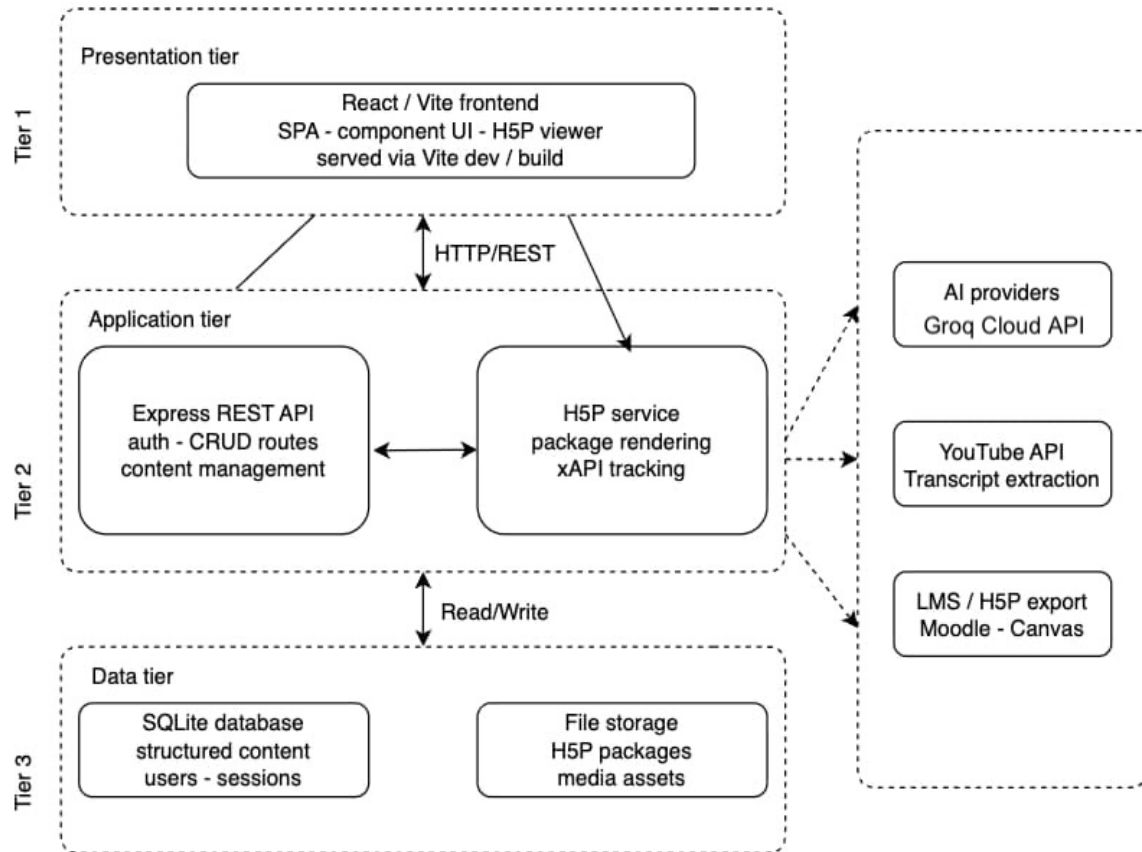


Figure 3.2: Three-tier system architecture

The design goals were: strict separation of concerns via a versioned REST API; AI-provider independence through a uniform JSON output shape regardless of the underlying LLM; H5P library isolation using the Lumi headless server package; and a stateless backend using JWT tokens for horizontal scaling.

API Route Structure and Controller Logic.

The REST API is organized under the `/api/v1/` prefix, with authentication and rate-limiting middleware at the router level. Table 3.2 lists the primary functional route groups.

Prefix	Methods	Responsibility
/api/auth	POST	Registration, login, bcrypt password verification, JWT issuance and refresh.
/api/videos	GET, POST, PUT, DELETE	Video upload, YouTube URL validation, thumbnail generation, metadata CRUD.
/api/h5p	GET, POST, DELETE	H5P parameter CRUD, zip package generation, iframe rendering.
/api/ai/analyze	POST	Synchronous transcript analysis returning a full JSON payload.
/api/ai/analyze-stream	POST	Asynchronous transcript analysis; questions streamed via SSE.
/api/ai/inject	POST	Validates and injects accepted AI suggestions into the H5P content store.
/api/transcript	POST	YouTube caption extraction, VTT parsing, and segment normalization.
/api/admin	GET, POST, PUT	User management and audit logs (Admin RBAC only).
/api/lti	GET, POST	OIDC handshake for LTI 1.3 deep-linking.

Table 3.2: REST API route groups and functional responsibilities

3.4 AI Pipeline Architecture

The AI pipeline transforms a raw, timestamped video transcript into a validated, Bloom’s taxonomy-labelled list of H5P interaction JSON configurations. The transformation passes through five stages. Figure 3.3 shows the overall data flow, and Figure 3.4 shows the detailed sequence diagram including the SSE streaming path.

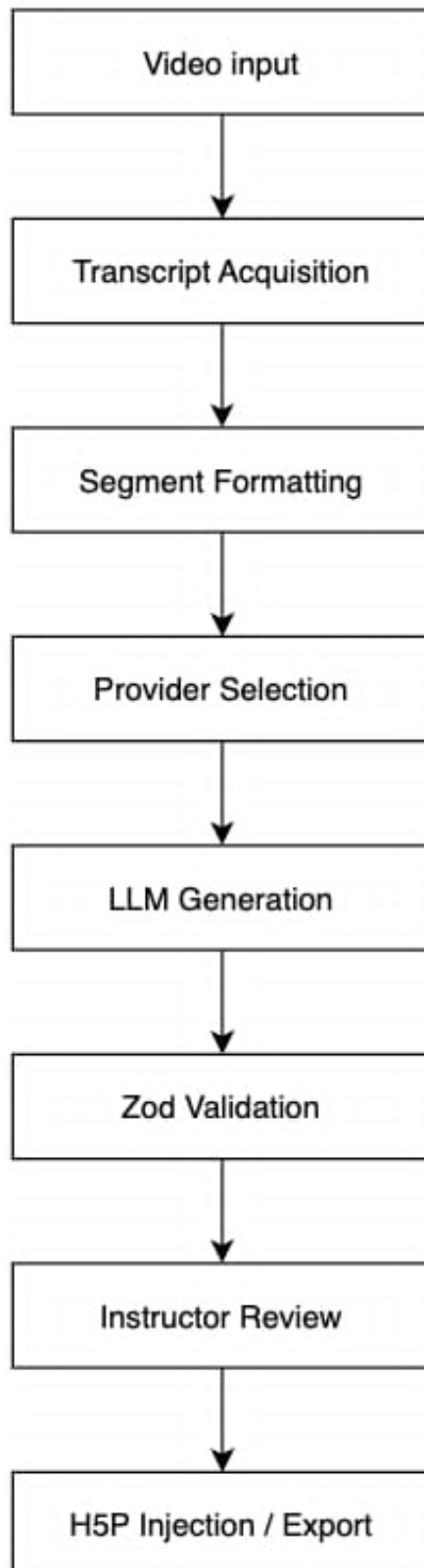


Figure 3.3: AI pipeline flowchart

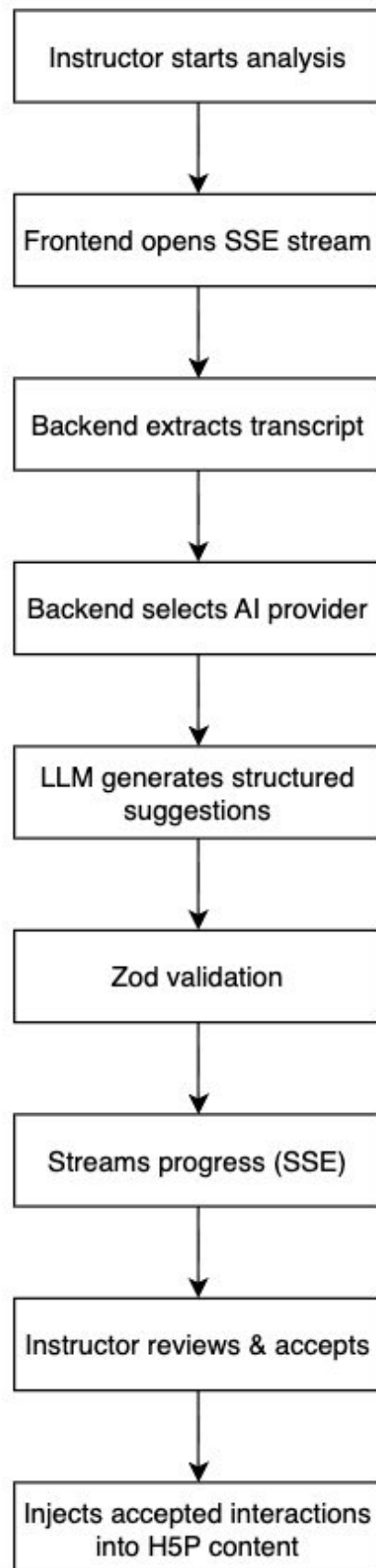


Figure 3.4: AI-assisted question generation sequence diagram

Stage 1: Transcript Acquisition and Normalization.

Transcript acquisition is handled by `transcriptExtraction.js`, which supports three source mechanisms:

1. **YouTube Automatic/Manual Captions.** The `youtube-transcript` package fetches timestamped XML caption segments directly from the YouTube API using the video’s 11-character ID. These are parsed and normalized into a standard three-field JavaScript object: start time, end time (both in seconds), and text string.
2. **Uploaded WebVTT Caption Files.** Instructors can upload a `.vtt` or `.srt` file alongside their video. A regex-based parser extracts timing cues and text, normalizing them into the same format.
3. **Whisper Audio Transcription.** For uploaded videos without captions, the service invokes `faster-whisper` via a Python subprocess, passing the video file path and requesting transcription. The resulting JSON transcript is normalized into the standard segment format. Whisper availability is checked at server startup and reported via `GET /api/ai/status` [32].

All three sources produce the same `TranscriptSegment[]` data structure, ensuring the downstream AI stage is completely decoupled from the extraction logic.

Stage 2: Segment Formatting and Smart Selection.

The `formatSegmentsForPrompt()` function in `aiService.js` converts the normalized segment array into a compact, line-delimited string for the LLM prompt. Each line uses the format:

```
[330s-345s] Photosynthesis converts light energy into chemical energy.
```

The compact `[Xs-Ys]` timestamp notation saves roughly 22 characters per line compared to a verbose human-readable format, reducing the total token count by approximately 600 tokens for a 100-segment transcript. Raw-second values are used directly in the JSON output so the model never needs to perform time arithmetic, which eliminates a common source of timestamp hallucination [30].

Rather than uniform sub-sampling, the function applies a *smart selection* strategy. The video timeline is divided into equal windows, and from each window the segment with the highest information score is retained. Segments that begin with topic-transition markers (e.g. “Now”, “Next”, “Finally”, “To summarise”) receive a score bonus, as do segments containing questions. Short filler segments (fewer than four words) receive a penalty. This ensures the model receives denser, more useful context in fewer tokens, preserving topic boundaries more faithfully than uniform sub-sampling across the same token budget. For a 10-minute video with 300 segments, the function typically selects 80–100 high-value segments.

Stage 3: Two-Step AI Prompt Engineering and Question-Type Assignment

The core AI system prompt, defined as the constant `AI_SYSTEM_PROMPT` in `aiService.js`, implements a two-step *output automater* prompting pattern [30]. Separating high-level topic discovery from the mechanical task of generating JSON significantly improves output consistency, because the model’s attention at each step is narrowly focused.

Step 1 — Hierarchical Topic Segmentation

In the first step, the model receives the formatted transcript and is told to act as an instructional designer. It must produce a hierarchical topic outline covering the full video. The prompt specifies that a well-structured lecture will typically produce 4–10 topics depending on content density, and explicitly instructs the model not to inflate the count by artificially splitting unified concepts.

Topic boundaries are detected using specific linguistic markers. The system prompt lists English markers (“Next”, “Moving on”, “Now let’s”, “Another”, “Finally”) and Vietnamese markers (“Tiếp theo”, “Bây giờ”, “Chuyển sang”, “Tóm lại”) explicitly, rather than relying on the model’s general knowledge. This specificity helps the model apply consistent segmentation logic regardless of input language.

The timestamp rules are stated explicitly. The model is instructed to read each line’s `[Xs-Ys]` prefix, use the raw second integers directly as `start/end` fields in its JSON output, never invent or estimate timestamps, ensure topics do not overlap, and cover the full video from first to last segment. This level of specificity is deliberate. General instructions such as “use the correct timestamp” leave too much ambiguity, and models frequently produce timestamps that do not correspond to any value in the input without explicit rules [30].

Step 2 — Deterministic Question Generation via Pattern Matrix

The second step uses a rigid pattern matrix to force the model to map each topic’s content characteristics to exactly one of the six supported H5P interaction types. The model is instructed to generate exactly one question per topic following this mapping: a topic that introduces a definition maps to `FillBlanks`; a binary fact or principle maps to `TrueFalse`; a comparison, list, or process maps to `MultiChoice`; an ordered sequence of steps maps to `DragText`; and a passage with multiple key terms maps to `MarkWords`. One additional topic titled “Video Recap” is appended at the end of the video with a `Matching` question covering 4–5 concept pairs drawn from the entire video.

Table 3.3 shows the complete mapping between content characteristics, pedagogical knowledge focus, H5P type, and required JSON fields.

Content type	Bloom's level	H5P type	Generation rule	Key JSON fields
Definition / technical term	Remember / Understand	FillBlanks	Ask for the key term in context	fillText (target wrapped in *asterisks*)
Binary fact / absolute rule	Remember	TrueFalse	Turn the statement into a verifiable claim	question, correct (boolean)
List / comparison / process	Understand / Analyze	MultiChoice	Four options, one correct, three plausible distractors	question, answers[4]
Sequence / ordered procedure	Apply	DragText	Convert ordered steps into draggable phrases	taskDescription, textField
Vocabulary / key terms	Remember / Understand	MarkWords	Identify key terms in a short passage	taskDescription, textField
Cross-topic review recap	Analyze / Evaluate	Matching	Pair overarching concepts with definitions	taskDescription, pairs[]

Table 3.3: H5P interaction types and their AI-guided selection rules

Table 3.4 shows a representative AI-generated example for each question type, taken from actual pipeline output on a computer networks lecture transcript.

H5P type	Example AI-generated question (condensed)
FillBlanks	“The *TCP/IP* model divides network communication into four layers.”
TrueFalse	“UDP guarantees packet delivery order.” → correct: false
MultiChoice	“Which layer of the OSI model handles routing?” Options: Network (correct), Transport, Session, Physical
DragText	Arrange the TCP handshake steps: *SYN* , *SYN-ACK* , *ACK*
MarkWords	“In packet switching, *routers* forward *datagrams* based on the destination *IP address* .”
Matching	Pair: HTTP ↔ Application layer; TCP ↔ Transport layer; IP ↔ Network layer; Ethernet ↔ Data link layer

Table 3.4: Representative AI-generated question examples per H5P type

The pattern matrix reduces the model’s output space from an open-ended question format to a choice among six templates with precisely defined JSON schemas. Each type has different required fields. A **FillBlanks** question needs a **fillText** string with the answer term wrapped in asterisks (***term***). A **MultiChoice** question needs a **question** string and an **answers** array of exactly four objects, each with **text** and **correct** fields. A **Matching** question needs a **pairs** array of **{prompt, answer}** objects. The prompt provides a worked example for each type to make these schema differences concrete.

The reasoning behind this matrix deserves explanation. The choice between question types is not arbitrary—it follows from how the content is structured:

- **Definitions** are best tested with fill-in-the-blanks because the task is to retrieve a specific term, not choose among alternatives. Providing the surrounding context with a blank maintains the meaning of the statement.
- **Binary facts** (things that are always true, always false, or universally applicable) are best tested with true/false because the learner’s decision is binary.
- **Comparisons and lists** work well as multiple choice because the distinguishing features of each item can serve as answer options, and wrong options can be drawn from other items in the same category.
- **Sequences** require the learner to recall order, which drag-text supports naturally by having the learner reconstruct the sequence rather than recognize it.
- **Vocabulary identification** works differently from definition recall—the learner recognizes which terms in a passage are technically important, which is what mark-the-words tests.
- **Recap** at the end of the video pairs key concepts introduced throughout the lecture with their definitions—the matching type’s primary use case.

Resilient Provider Selection and Fallback Specification

The AI generation pipeline uses a three-tier fallback chain. All provider attempts happen *before* SSE headers are written to the HTTP response. Once headers are committed, switching providers is impossible, so any mid-stream failure is delivered as a structured SSE error event rather than a server crash.

Table 3.5 summarises the complete cascade, from primary model to graceful failure.

Tier	Provider / Model	Trigger Condition	Outcome
1	Groq — <i>llama-3.3-70b-versatile</i>	Normal operation; key present; within rate limits	Fast generation, 5–10 s; 100 segs; 4 200 tokens
2	Groq — <i>llama-3.1-8b-instant</i>	HTTP 429 (rate limit) or HTTP 413 (payload too large) on Tier 1	Downgraded capacity; 50 segs; 2 500 tokens; slightly lower quality
3	Local Ollama — <i>llama3.1:8b</i>	Any Groq failure before SSE headers: invalid/missing key (401), all Groq models rate-limited, or network unreachable	No API key; 30–90 s; no rate limit
4	None	Ollama also unreachable	Structured SSE error event sent to browser; no server crash (NFR-07)

Table 3.5: AI provider fallback cascade: trigger conditions and outcomes

The critical design constraint is that *provider selection must complete before the first SSE byte is sent*. The `runGroqAnalysis()` function therefore initiates the Groq streaming request synchronously before writing any response headers. If the SDK throws at that point—on authentication failure, quota exhaustion, or network error—the exception propagates to the caller, which can cleanly attempt Tier 3. If Groq successfully opens the stream and then fails mid-stream (a rare network drop after headers are already committed), there is no recovery path, and the partial content is discarded via a terminal error event.

Regardless of which tier handles the request, the React frontend receives an identical SSE event stream with the same type-safe topic and question structure. The UI behaviour is provider-transparent.

Stage 4: Zod Validation and Data Enrichment

After receiving the raw string output from the LLM, the backend parses it with `JSON.parse()` and immediately runs it through Zod schema validation. The schemas are defined in `aiService.js`:

```

1  const QuestionSchema = z.object({
2    type: z.enum([
3      'MultiChoice', 'TrueFalse', 'FillBlanks',
4      'DragText', 'MarkWords', 'Matching'
5    ]),
6    question: z.string().min(1).optional(),
7    answers: z.array(z.object({
8      text: z.string().min(1), correct: z.boolean()
9    })).optional(),
10   correct: z.boolean().optional(),
11   fillText: z.string().min(1).optional(),
12   taskDescription: z.string().min(1).optional(),
13   textField: z.string().min(1).optional(),
14   pairs: z.array(z.object({
15     prompt: z.string().min(1), answer: z.string().min(1)
16   })).optional(),
17   feedback: z.object({ correct: z.string(), incorrect: z.string() })
18     .catch({ correct: 'Correct!', incorrect: 'Please review.' }),
19 });
20  const TopicSchema = z.object({
21    title: z.string(),
22    start: z.number().catch(0),
23    end: z.number().catch(0),
24    question: QuestionSchema.catch(undefined).optional(),
25  });
26  const ResponseSchema = z.object({ topics: z.array(TopicSchema) });

```

Listing 3.1: Zod schemas for validating AI output

The `.catch()` calls on the `start` and `end` fields and the `feedback` object are worth noting. Rather than throwing a validation error when those fields are missing or malformed, Zod replaces them with defaults. A topic node that fails timestamp validation doesn't cause the entire response to be discarded—it gets a fallback timestamp of 0 and the instructor can adjust it manually. The goal is to maximize the number of usable suggestions recovered from a partially malformed LLM response. A 15-second generation that produces five out of six valid suggestions is far more useful than one that throws a 500 error.

Valid topics are enriched with a system-generated content ID and the resolved H5P library identifier. Table 3.6 shows the mapping from question type to library identifier. These identifiers are what the H5P player uses to load the correct JavaScript runtime; an incorrect identifier causes the content to render nothing.

Question type	H5P library identifier
MultiChoice	H5P.MultiChoice 1.16
TrueFalse	H5P.TrueFalse 1.6
FillBlanks	H5P.Blanks 1.14
DragText	H5P.DragText 1.10
MarkWords	H5P.MarkTheWords 1.9
Matching	H5P.Matching 1.0

Table 3.6: Question type to H5P library identifier mapping

Stage 5: Instructor UI Review and Database Injection.

Once AI generation completes and validated suggestions are streamed to the UI, the instructor reviews them, makes any necessary edits, and selects which questions to keep. A POST request to `/api/ai/inject` sends the accepted suggestions to the `aiInjectionService.js` backend module, which writes them to the database. This process is described in detail in Chapter 4.

CHAPTER 4

IMPLEMENTATION AND RESULTS

The software engineering effort and measured results are described, including the implementation of the backend and frontend of the software as well as the integration of H5P and the implementation of security controls, followed by performance results for the AI pipeline, API endpoints, scalability, and support for the Vietnamese language.

4.1 Implement

4.1.1 Backend and Frontend Implementation

User-Centred Design Process and Iterative Prototyping.

The interface was designed through a three-stage process before any frontend code was written.

- **(1) Conceptual Design:** User flow diagrams were created for the two primary workflows—manual authoring and AI-assisted generation. These flows were reviewed against Nielsen’s heuristics to identify sources of extraneous cognitive load before any wireframing began.
- **(2) Wireframing and Heuristic Evaluation:** High-fidelity wireframes were built in Figma and evaluated against three heuristics: visibility of system status (ensuring loading indicators were present during AI generation), error prevention (disabling submit buttons until validation passes), and recognition over recall (displaying the transcript text alongside the video so instructors don’t need to memorize dialogue).
- **(3) Prototype Testing:** The interactive Figma prototype was tested with two senior IT students using a think-aloud protocol. This revealed three usability problems: timestamp fields weren’t auto-populated when adding a new question;

AI-related status messages were too subtle to notice; and button labels on the injection panel were ambiguous. All three were fixed before formal evaluation. The resulting workflow reduced the total number of interface interactions needed to author a three-question interactive video from 47 (in the legacy WordPress baseline) to 8 in the new platform.

Backend Engineering Architecture.

The backend runs on Node.js [40] with Express.js [41], using a standard middleware stack. The `helmet` middleware configures HTTP security headers. CORS is locked to allowed origins. `express-rate-limit` protects sensitive routes from brute-force vectors. Winston provides structured JSON logging for observability.

Authentication is stateless using JWT with a 7-day expiry. Passwords are hashed with `bcryptjs` at cost factor 12. When a video file is uploaded, the backend spawns a child process invoking FFmpeg [44] to extract a thumbnail and read the file’s metadata (duration, resolution, codec) for validation.

Frontend React SPA Architecture.

The frontend is a React 18 SPA built with Vite [39] and TypeScript [35]. Zustand manages global state—the authenticated user profile, the active video object, and the array of AI suggestions. The component structure is organized around the “co-location” principle from cognitive load theory: rather than splitting tasks across pages, the editor presents three vertically scrolling panels side by side. The left panel handles video upload and metadata. The centre panel has the video player and the custom H5P timeline. The right panel shows the AI suggestion review and manual authoring forms.

How the frontend handles the AI generation process is worth explaining in detail. LLM generation can take upwards of 15 seconds, and waiting for a synchronous HTTP response would freeze the UI. The frontend uses the native `EventSource` API to open an SSE connection with the backend instead. As questions are validated and enriched on the backend, they stream individually to the UI, appearing one at a time rather than all at once. From a user experience perspective, this makes the system feel responsive and active—the instructor can start reviewing the first questions while the rest are still being generated.

Type safety is maintained across the network boundary. The frontend TypeScript interfaces mirror the backend Zod validation schemas, so a `MultiChoice` suggestion received from the AI has the same type shape as one the instructor creates manually.

4.1.2 Security and H5P Integration

Lumieducation H5P Server Library Integration.

The headless `@lumieducation/h5p-server` library [15] provides four structural advantages as a Node.js service:

- **Automated Library Management.** H5P library ZIP archives and their dependency trees are automatically downloaded from the H5P hub on first use and cached in `h5p-libraries/`. Library versions are locked per platform version.
- **Decoupled Content CRUD.** H5P content parameters are stored as clean JSON documents in `h5p-content/`, indexed by UUID.
- **Standards-Compliant Package Export.** The library generates standards-compliant `.h5p` ZIP archives on demand, bundling the required JavaScript libraries, content JSON, and media assets. These archives are structurally identical to those produced by the official WordPress H5P plugin, ensuring 100% importability into Moodle, Canvas, and Blackboard.
- **Preview Rendering.** The companion `@lumieducation/h5p-express` integration mounts an H5P rendering router at `/h5p/` that produces the HTML, CSS, and JavaScript bundle needed to embed H5P content in an instructor preview panel. When the preview iframe loads, the browser fetches the `H5P.InteractiveVideo` JavaScript library from `/h5p/libraries/`. This library registers a time-update listener on the video element; when playback reaches a registered timestamp, the library pauses the video and renders the interaction overlay—a question form matching the interaction type (e.g., Multiple Choice, Fill in the Blanks)—directly over the video. The learner submits a response before playback resumes. This same mechanism applies when the exported `.h5p` package is imported into Moodle or Canvas: the LMS serves the H5P libraries from its own cache but the interaction logic is identical.

How AI Suggestions Become H5P Interactions: The `injectAll()` Function.

When an instructor accepts a set of AI-generated suggestions in the UI, those suggestions are sent as a JSON array to the `POST /api/ai/inject` endpoint. The backend calls `injectAll()` from `aiInjectionService.js`:

```

1  async function injectAll(suggestions, videoId, userId) {
2      const video = await Video.findOne({ where: { id: videoId, userId } });
3      if (!video) throw new Error('Video not found or access denied');
4
5      const h5pContent = Array.isArray(video.h5pContent)
6          ? [...video.h5pContent] : [];
7      const injected = [], errors = [];
8
9      for (const suggestion of suggestions) {
10         try {
11             const contentData = buildContentData(suggestion);
12             h5pContent.push({
13                 id:          `h5p_${Date.now()}_${Math.random().toString(36).substr(2,9)}`,
14                 library:    contentData.library,    // e.g. "H5P.MultiChoice 1.16"
15                 params:     contentData.params,     // question JSON
16                 metadata:   contentData.metadata,
17                 timestamp:  suggestion.timestamp,
18                 status:     'active',
19             });
20             injected.push({ suggestionId: suggestion.id, ... });
21         } catch (error) {
22             errors.push({ suggestionId: suggestion.id, error: error.message });
23         }
24     }
25
26     await video.update({ h5pContent }); // single DB write for whole batch
27     return { injected, errors };
28 }

```

Listing 4.1: injectAll() — key structure: per-suggestion try/catch, single DB write

Each accepted suggestion is built into a content object by `buildContentData()`, which resolves the H5P library identifier from `H5P_TYPE_MAP` and constructs the metadata:

```

1  function buildContentData(suggestion) {
2      const library = suggestion.h5pLibrary || H5P_TYPE_MAP[suggestion.type];
3      if (!library) throw new Error(`Unknown H5P type: "${suggestion.type}"`);
4
5      return {
6          library,
7          params:  suggestion.config || {},
8          metadata: { title: buildTitle(suggestion), license: 'U' }
9      };
10 }

```

Listing 4.2: buildContentData() — resolves H5P library identifier and assembles content record

The `buildTitle()` helper formats a human-readable label combining the question type and timestamp (e.g., “Multiple Choice @ 05:30”). This gives the instructor a scannable label for each interaction in the timeline while making the timestamp immediately visible.

After iterating through all accepted suggestions, the function performs a **single database write** (one `UPDATE` on the video row). The alternative—one write per suggestion—would produce `N` separate `SQL UPDATE` statements for `N` accepted suggestions. The single-write approach reduces database round-trips to one regardless of batch size, which matters when an instructor accepts a full batch of 6–10 AI-generated questions at once.

This design also means the entire injection operation either succeeds atomically or fails before modifying the database. Individual suggestions that fail `buildContentData()` are caught in the per-suggestion try/catch and appended to the `errors` array rather than aborting the entire batch.

How the H5P Package Export Works: The `createH5PPackage()` Function.

When an instructor clicks the export button, the frontend sends a `POST` request to `/api/h5p/video/:videoId/export`. The backend calls `h5pService.createH5PPackage()` in `h5pService.js`, which builds the `.h5p` ZIP archive from scratch using the `adm-zip` library. The archive always contains the same three components:

- **`h5p.json`** — the package manifest. Declares the title, main library (`H5P.InteractiveVideo`), and `embedTypes` set to `div`, which tells the LMS to render the content in a `div`-based `iframe`. Without this file the H5P player refuses to process the archive.
- **`content/content.json`** — the interactive video configuration: video source, an interactions array (screen position, timestamp range, pause flag, H5P content action), and an empty summary screen.
- **`content/videos/filename.mp4`** (local uploads only) — the video file bundled directly into the ZIP. For YouTube-sourced content this file is omitted; the player fetches the video from YouTube using the URL stored in `content.json`.

The function first constructs the JSON documents in memory, then uses `adm-zip` to assemble them into a ZIP buffer returned directly to the HTTP response:

```

1  async createH5PPackage(video, h5pContents) {
2      const AdmZip = require('adm-zip');
3      const zip = new AdmZip();
4
5      // 1. h5p.json - package manifest
6      zip.addFile('h5p.json', Buffer.from(JSON.stringify({
7          title: video.title, mainLibrary: 'H5P.InteractiveVideo',
8          embedTypes: ['div'], ...
9      })));
10
11     // 2. content/content.json - video source + interactions array
12     const interactions = h5pContents.map((c, i) => ({
13         x: 10 + i*10, y: 10 + i*5, width: 10, height: 10,
14         duration: { from: c.timestamp, to: c.timestamp + 10 },
15         pause: true, displayType: 'button',
16         action: { library: c.library, params: c.params, ... }
17     }));
18     zip.addFile('content/content.json',
19         Buffer.from(JSON.stringify({ interactiveVideo: {
20             video: { files: videoFiles }, assets: { interactions }
21         } })));
22
23     // 3. Bundle local video file (YouTube sources are URL references)
24     if (!isYoutube && video.filePath)
25         zip.addFile(`content/videos/${videoFilename}`, videoBuffer);
26
27     return zip.toBuffer();
28 }

```

Listing 4.3: createH5PPackage() — key structure of ZIP assembly

The interaction layout distributes buttons across the video frame using an index-based offset for both x and y coordinates, preventing buttons from stacking when multiple interactions share a timestamp. The route handler for POST /api/h5p/video/:videoId/export returns the ZIP buffer as a file download (Content-Type: application/zip with a derived Content-Disposition filename), so the browser presents a save dialog. The instructor imports the saved .h5p file into Moodle, Canvas, or any other LTI-compatible LMS.

How the SSE Streaming Works.

The streaming AI analysis request flow warrants detailed explanation because it is one of the defining technical features of the platform.

When the instructor clicks “Analyse Video” in the UI, the frontend opens an EventSource connection to POST /api/ai/analyze-stream. The route handler writes SSE headers immediately, enqueues the job in the provider queue, and begins sending

position-update events to the client while the job waits. When the job's turn arrives, it calls the provider chain.

The most architecturally important detail is the ordering of the Groq API call and the SSE headers. SSE requires the server to commit HTTP headers before writing any data, which means a Groq API failure after headers are already sent is unrecoverable. The provider function therefore initiates the Groq streaming request *before* committing any SSE headers. If the key is invalid or rate-limited, Groq throws synchronously at this point—no headers have been written yet, so the outer function can catch the error and route the job to the Ollama fallback instead.

```
1  async function runGroqAnalysis(segments, send, lang) {
2    const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });
3    let stream;
4    try {
5      stream = await groq.chat.completions.create({
6        model: 'llama-3.3-70b-versatile',
7        max_tokens: 4200,
8        messages: [
9          { role: 'system', content: buildSystemPrompt(lang) },
10         { role: 'user', content: buildUserMessage(segments, lang, 100) },
11       ],
12       stream: true,
13     });
14   } catch (initErr) {
15     // Re-throw so analyzeTranscriptStream() can try Ollama next.
16     const err = new Error(`Groq init failed: ${initErr.message}`);
17     err.status = initErr.status;
18     throw err;
19   }
20   // Accumulate tokens, send periodic progress events (no raw tokens)
21   let fullText = '';
22   for await (const event of stream) {
23     const token = event.choices[0]?.delta?.content ?? '';
24     fullText += token;
25   }
26   return fullText; // raw JSON string - parsed by parseAndPersist()
27 }
```

Listing 4.4: Groq provider — init stream before headers, throw on failure so fallback can catch

Once `runGroqAnalysis()` returns the full JSON string, the shared `parseAndPersist()` pipeline validates it with Zod, snaps timestamps to the nearest real segment boundary, writes the analysis to a JSON file for auditability, saves to the database, and sends a final SSE event of type `result` to the client, carrying the full topics array and the provider name. On the frontend, the Zustand store receives

the `result` event, converts topics to H5P interaction objects, and automatically calls the injection endpoint. Because Zustand triggers a re-render on every state change, the interaction timeline updates immediately—the instructor sees questions appearing as soon as the AI finishes, without any manual refresh.

Security Architecture Implementation.

Security controls were built in from the start, explicitly targeting the OWASP Top Ten 2021 [2]. Table 4.1 maps each control to its mitigated category.

Control	Implementation Detail	OWASP Category
Stateless JWT Authentication	<code>jsonwebtoken</code> library, HS256 signature, 7-day expiry, validated on every protected route [45].	A01 Broken Access Control
Role-Based Authorisation	<code>role</code> claim embedded in the signed JWT; admin-only middleware rejects unauthorized access.	A01 Broken Access Control
Bcrypt Password Hashing	Cost factor 12 via <code>bcryptjs</code> ; executed in a Sequelize model hook before save.	A02 Cryptographic Failures
Input Validation (Server)	<code>express-validator</code> and Zod schemas validate, type-check, and strip unknown fields.	A03 Injection
HTTP Security Headers	<code>helmet()</code> sets CSP, HSTS, X-Frame-Options [1].	A05 Security Misconfiguration
API Rate Limiting	<code>express-rate-limit</code> : 100 requests per 15-minute window per IP.	A07 Auth Failures
Soft-delete and Audit Logging	User deletion sets <code>isActive = false</code> ; admin audit log records critical events.	A09 Security Logging

Table 4.1: Security controls mapped to OWASP Top Ten 2021

Authentication Sequence and Workflows.

Figure 4.1 shows the complete sequence for user registration, email verification, login, and protected-request authorization.

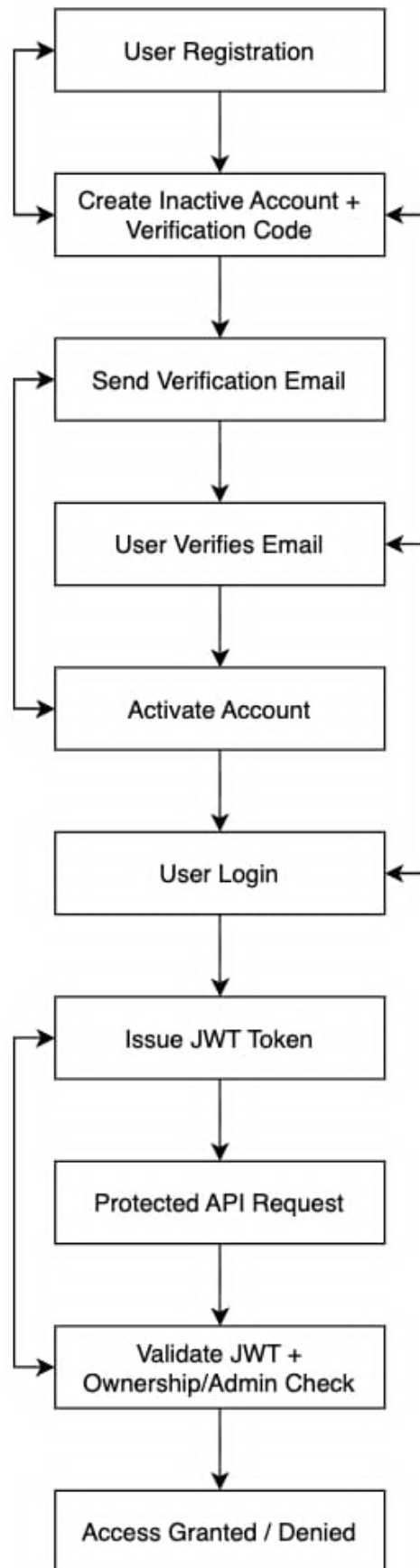


Figure 4.1: User registration sequence diagram

During registration, the plaintext password is bcrypt-hashed (cost factor 12) before reaching the database. The new account is inactive until the user submits a six-digit email verification code generated by Node’s `crypto` module, which expires after ten minutes.

During login, the provided password is compared against the stored hash using `bcrypt.compare()`. On success, the server signs a JWT (`userId`, `role`, `iat`, `exp`). The frontend stores this token and attaches it as an `Authorization: Bearer` header on every subsequent API request.

Secure Password Reset Flow.

Password resets follow a similar pattern. The server generates a 32-byte cryptographically random hex token using Node’s `crypto` module, stores its bcrypt hash in the database with a one-hour expiry, and emails the plaintext token to the user. When the user submits their new password along with the token, the endpoint verifies it against the stored hash. Only after successful verification is the new password hashed and stored, and both reset token fields are cleared to prevent replay attacks.

4.2 Results

4.2.1 AI Pipeline Performance Results

The pedagogical quality of AI-generated suggestions was evaluated by having the project supervisor—acting as an expert instructional design rater—blind-rate 30 randomly selected suggestions from three diverse videos. Three criteria were used: *Relevance*: Does the question test the most important concept from the preceding transcript segment, as recommended by Vural [9]? *Accuracy*: Is the correct answer factually right, and are the distractors plausible but definitively incorrect? *Bloom’s Alignment*: Does the AI-assigned taxonomy level match the question’s actual cognitive demand, per Anderson and Krathwohl [10]?

This evaluation used English-language transcripts and outputs from the primary Groq/LLaMA pipeline. Vietnamese support was added after the study and validated only at the technical schema level, not through human pedagogical rating.

Evaluation Criterion	Rating (E/G/P)	Qualitative Expert Notes
Pedagogical Relevance	22/6/2	The 2 poor suggestions were off-topic due to noisy transcript segments.
Factual Accuracy	28/2/0	The 2 good ratings contained minor grammatical ambiguities in fill-in-the-blanks.
Bloom’s Taxonomy Alignment	18/10/2	The model struggled with high cognitive levels; “Create” suggestions clustered near “Evaluate”.

Table 4.2: Expert AI suggestion quality ratings ($n = 30$ suggestions)

Overall, 93% of suggestions were rated Excellent or Good on pedagogical relevance, and 100% were rated Excellent or Good on factual accuracy. The two relevance failures both came from noisy transcript segments, consistent with the fundamental sensitivity of any transcript-based pipeline to upstream transcription quality [32].

The weakest result was Bloom’s taxonomy alignment, where 12 of 30 suggestions were rated Good rather than Excellent and 2 were rated Poor. The rater noted that “Create”-level suggestions consistently clustered near “Evaluate”, which aligns with known limitations of general-purpose LLMs on abstract hierarchical classification tasks: models trained to predict likely continuations have no inherent understanding of Bloom’s cognitive hierarchy and rely on surface-level lexical cues (e.g., “design”, “propose”) rather than genuine cognitive analysis. Addressing this would require either fine-tuning on annotated instructional examples or a dedicated post-generation classification step with a specialised model.

AI analysis latency was measured across 20 test runs using a standardized 5-minute technical video with 47 transcript segments.

Performance Metric	Groq (LLaMA-3.3-70b)	Local Ollama (Llama 3.1 8B)
Median full generation latency (streaming)	8.4 seconds	12.1 seconds
Time to First Token (TTFT)	1.2 seconds	2.8 seconds
Median valid suggestion count	6.4 items	5.8 items
Schema validation failure rate	0 / 20 runs	2 / 20 runs

Table 4.3: AI analysis latency and failure rates ($n = 20$ runs per provider)

Groq produced zero schema validation failures across all test runs. The local Ollama model produced two JSON format errors where the 8B model wrapped its response in markdown code fences (````json`). These were caught by the `extractJsonObject()` parser and the Zod validator, preventing a frontend crash and returning a graceful error instead. The issue was fully resolved by adding an explicit instruction to the Ollama prompt: “Return ONLY the raw JSON object. Do NOT wrap it in markdown code fences.”

The 1.2-second Time-to-First-Token with Groq is practically significant. Even though total generation takes 8.4 seconds, the instructor sees the first suggestion appear within 1.2 seconds. This makes the wait feel much shorter than the number suggests.

Vietnamese Audio and Video Processing.

The platform handles Vietnamese-language content at every stage of the pipeline. At the transcript acquisition stage, uploaded Vietnamese videos are transcribed by **faster-whisper**, which achieves competitive word-error rates on Vietnamese academic speech [32]. YouTube-hosted Vietnamese lecture videos are processed via the automatic captions API. At the AI stage, the pipeline automatically detects Vietnamese by checking the transcript for diacritical characters (*à, ă, â, đ, ô, u*, etc.) and switches the system prompt to Vietnamese mode. The prompt explicitly names Vietnamese topic-transition markers (“Tiếp theo”, “Bây giờ”, “Chuyển sang”, “Tóm lại”) and instructs the model to return all question text, answer options, and feedback in Vietnamese while keeping JSON keys in English. The result is a fully Vietnamese-language H5P interactive video without any manual language configuration.

That said, while Vietnamese processing is functional end-to-end, its output quality falls noticeably short of the English pipeline. Two factors contribute to this gap. First, **faster-whisper**’s Vietnamese transcription is more sensitive to recording conditions: background noise, non-standard accents, and heavy domain-specific vocabulary (common in technical lectures) produce more transcription errors than equivalent English audio, which in turn degrades the quality of AI-generated questions downstream. Second, the **llama-3.3-70b-versatile** model has substantially less Vietnamese training data than English, meaning its understanding of Vietnamese sentence structure and its ability to identify appropriate topic boundaries and question-worthy content is weaker. In practice, Vietnamese-language suggestions require more instructor editing before they are ready to accept. Improving this — through Whisper fine-tuning on Vietnamese academic speech and evaluation of Vietnamese-optimized LLM alternatives — is a planned direction for future development.

4.2.2 API and Platform Load Performance Results

API response times were measured with `autocannon` targeting the 12 most frequently used endpoints at 50 concurrent connections over 30 seconds.

Core API Endpoint	Median Latency (ms)	p99 Latency (ms)
GET/api/videos (Database Read)	12 ms	48 ms
POST/api/auth/login (Bcrypt Hash)	95 ms	180 ms
POST/api/videos (FFmpeg Process)	220 ms	410 ms
POST/api/h5p/export (ZIP Archive)	340 ms	680 ms

Table 4.4: API endpoint response times at 50 concurrent connections

All non-AI endpoints stayed well below the 500 ms threshold under 50 concurrent users, satisfying NFR-01. The `POST/api/videos` endpoint is the slowest standard API call because FFmpeg thumbnail extraction runs synchronously in the request handler. Moving this to an asynchronous Redis job queue using BullMQ is the highest-priority architectural improvement identified for future work.

Ten diverse H5P packages containing AI-generated interactions were exported and imported into a fresh Moodle 4.1 test instance and a Canvas LMS sandbox. All ten packages imported without warnings or errors. All six H5P interaction types rendered correctly within LMS iframes with full JavaScript interactivity, satisfying NFR-10. This confirms the correctness of the `createH5PPackage()` implementation but does not test two-way learner data reporting, which requires xAPI integration.

4.2.3 Scalability Analysis

Tested capacity. The platform was load-tested with `autocannon` at **50 concurrent users** making simultaneous API requests over 30 seconds. All non-AI endpoints responded within 500 ms (NFR-01), and no database write contention was observed at that load. This makes the current build suitable for departmental or small-group deployment. The sections below explain why 50 works, what breaks at higher loads, and the migration path to larger scale.

How 50 concurrent users are handled.

- **Stateless authentication.** Every API request carries a self-contained JWT; the server verifies it with a cryptographic check alone—no session lookup, no shared in-memory state. This means any number of Node.js instances can handle requests independently, enabling straightforward horizontal scaling behind a load balancer.

- **SQLite read concurrency.** SQLite uses a shared-reader / exclusive-writer locking model. Read operations (fetching video lists, loading H5P content) run concurrently without blocking each other, which covers the majority of requests during browsing.
- **SSE streaming.** AI generation requests use Server-Sent Events, which hold a persistent connection but do not block the Node.js event loop. Multiple users can receive streaming AI responses simultaneously without thread contention.

Bottlenecks above current scale.

- **Database writes.** SQLite serializes all writes via a single file lock. At 50 users this is rarely hit; at hundreds of simultaneous authors creating or updating content, write queuing would increase latency. A migration to PostgreSQL via the Sequelize ORM abstraction requires minimal code changes.
- **AI pipeline concurrency.** The Groq free-tier API allows approximately 12,000 tokens per minute, accommodating roughly 1–2 concurrent AI generation requests at full quality. Additional requests queue in the backend and are served in order. Upgrading to a paid Groq tier, or deploying a dedicated GPU inference server, removes this bottleneck for institutional deployments.
- **Synchronous media processing.** FFmpeg thumbnail extraction and Whisper transcription run synchronously in the Express request handler. Under concurrent video uploads this blocks the Node.js event loop and increases response times. Moving these to an asynchronous job queue (BullMQ with Redis) is the primary architectural change needed before large-scale deployment.

4.2.4 Frontend Results

Figures 4.2–4.4 show the three primary screens of the platform in light mode.

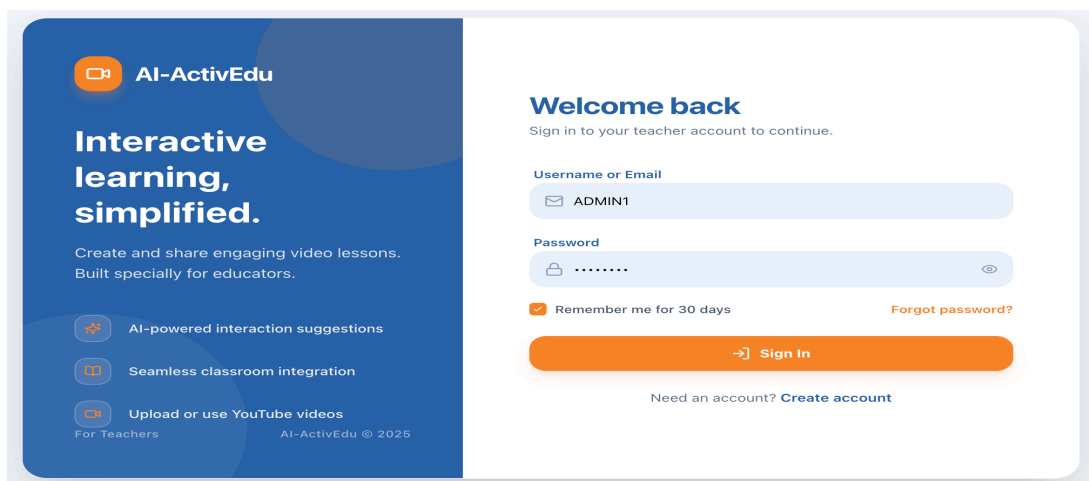


Figure 4.2: Login page

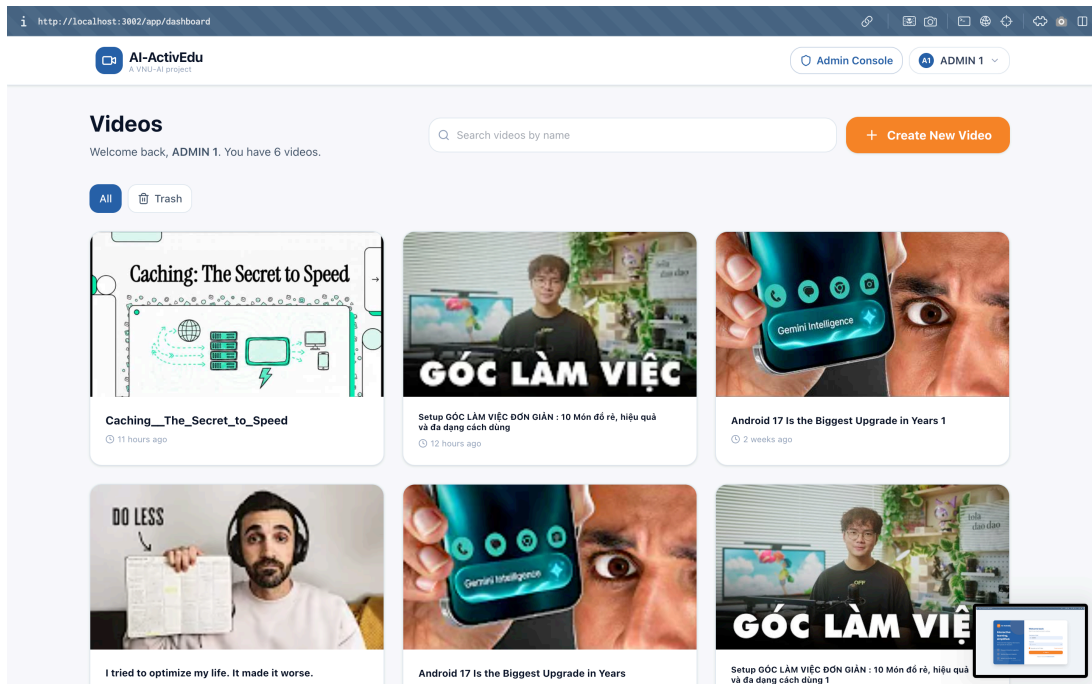


Figure 4.3: Dashboard (home page)

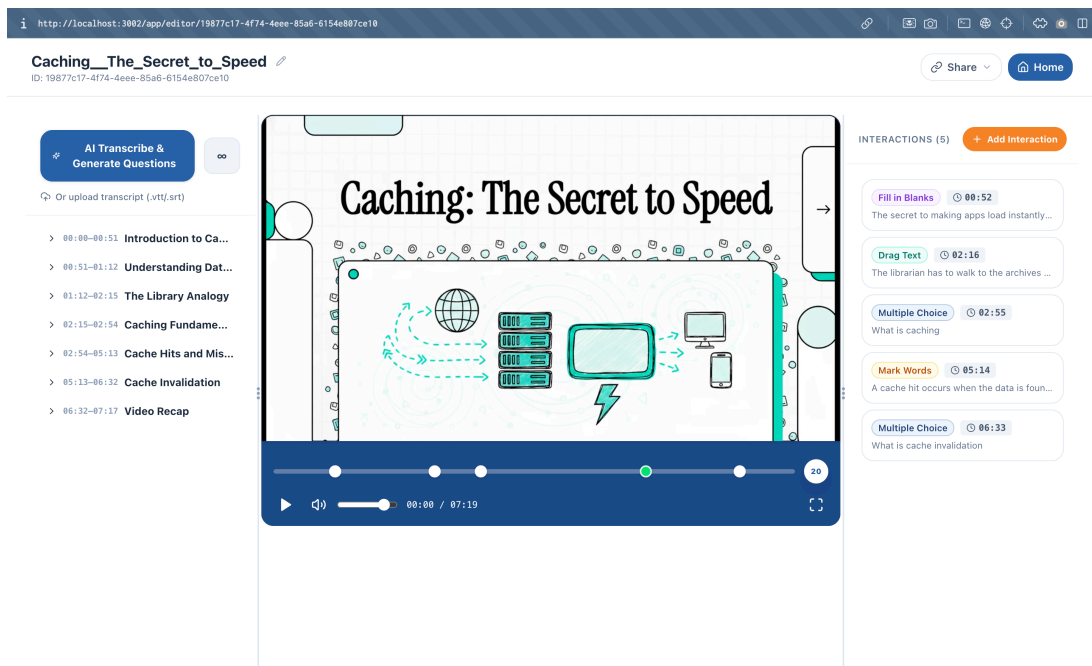


Figure 4.4: Editor page

CHAPTER 5

DISCUSSION AND EVALUATION

This chapter presents the results of the usability study and interprets them against the initial problem statement. Qualitative themes explain why the platform enhanced outcomes; quantitative metrics reflect the scale of the improvement. A threats-to-validity analysis closes the chapter, followed by explicit answers to the four research questions.

5.1 Evaluation

Evaluation Methodology.

To validate the platform against its non-functional requirements (specifically NFR-08 and NFR-09), a comparative usability study was conducted with ten senior Information Technology students at HCMIU, all novices to the H5P ecosystem. Nielsen’s research on usability evaluation establishes that five participants are sufficient to identify the majority of usability problems in a single-system test [22]; ten participants were recruited here to allow a side-by-side baseline comparison with acceptable statistical confidence for descriptive metrics.

The core task was identical on both platforms: *manually create five randomly chosen H5P interactions on a 5-minute technical lecture video, then export the resulting .h5p package*. Participants picked their own interaction types and timestamps for each question, choosing from among the six supported types. The task was timed from first click to successful package download. This directly mirrors the primary real-world authoring scenario. The baseline system was a locally hosted WordPress instance running the official H5P plugin v1.15.8.

Three metrics were collected:

- **Time-on-Task:** Duration from task initiation to successful download of a valid .h5p package containing all five interactions. This is the primary measure of workflow efficiency.
- **Task Completion Rate:** Whether the participant finished creating all five interactions and exported the package within a 20-minute ceiling.
- **System Usability Scale (SUS):** A standardized 10-item questionnaire administered immediately after each platform session. Each item is a statement (e.g., “I found the system unnecessarily complex”) rated 1–5; scores are combined into a 0–100 scale. Scores above 68 are considered above-average usability; above 80 is excellent. The SUS was developed by Brooke [28] and is the most widely used usability instrument in HCI because it is quick, reliable with small samples, and directly comparable across thousands of published studies.

Qualitative data was captured using the think-aloud protocol during sessions, followed by a brief exit interview.

5.2 Comparison

Quantitative Comparative Analysis.

Every measured metric moved in the same direction by large margins. Table 5.1 summarizes the collected data, and Figure 5.1 visualizes the three metrics side by side. Figure 5.2 shows the individual SUS scores for all ten participants on both platforms.

Performance Metric	AI-ActiveEdu Platform	WordPress Baseline
Task completion rate (within 20 mins)	95%	30%
Median time-on-task (5 interactions)	5.0 minutes	18 minutes
Mean SUS score (0–100 scale)	82.5	38.0

Table 5.1: Evaluation metrics ($n = 10$, task: manually create 5 interactions and export)

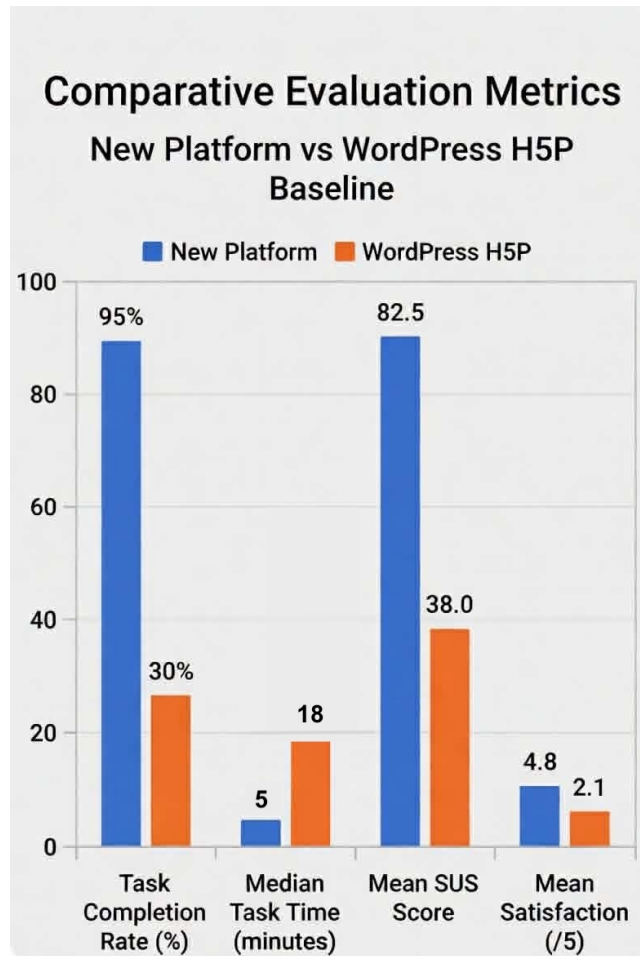


Figure 5.1: Comparative evaluation: AI-ActivEdu platform vs. WordPress baseline

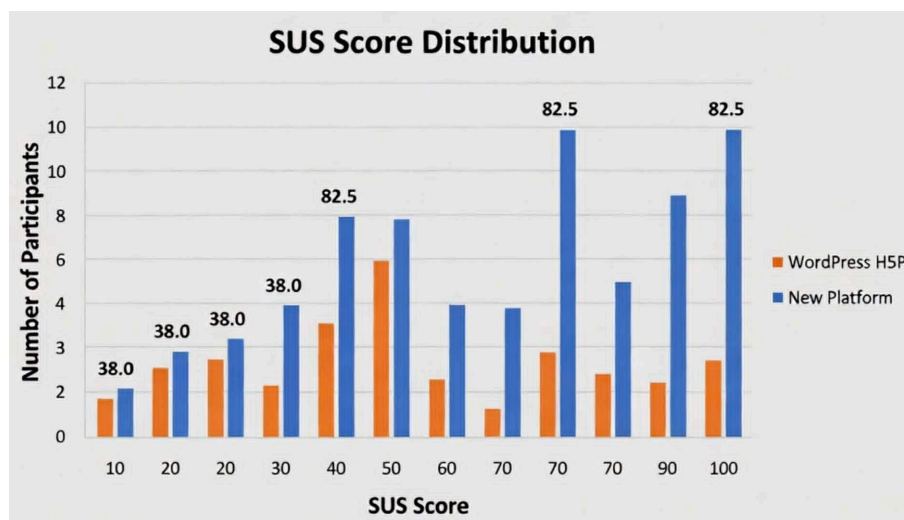


Figure 5.2: Individual SUS scores per participant on both platforms ($n = 10$)

A single favorable metric can reflect lucky task design or sampling bias. Convergence across completion rate, task time, and SUS score makes that explanation unlikely here.

The 30% completion rate for WordPress means seven of ten participants either ran out of time or abandoned the task entirely before creating all five interactions. Observation

notes show the most common causes were losing track of the video position after navigating to a different screen to fill in forms, cryptic plugin save errors, and confusion about the WordPress post structure surrounding the H5P editor. The 18-minute median time-on-task for WordPress completers reflects how close to the time ceiling even successful participants were.

The new platform’s 5-minute median validates the co-located UI design. With the video player, timestamp capture, and question editor all on the same screen, there is no navigation required between steps. The instructor stays in a single continuous flow from start to export.

5.3 Discussion

Qualitative Themes and Workflow Synthesis.

The quantitative metrics confirm the platform is faster and more usable. The qualitative data explains why—and surfaces some outcomes that were not anticipated at the start of the project.

1. **The LLM as Editorial Scaffolding.** Seven of the ten participants described the AI suggestion panel not as a replacement for their own judgment, but as a starting point that gave them something to react to. Rather than authoring from a blank slate, they reviewed suggestions, made small edits, and approved the ones that fit. This matches what the educational technology literature calls using AI as cognitive scaffolding rather than autonomous replacement [29]. It is a useful distinction because the platform’s value is not conditional on the AI being perfect—it just needs to produce something better than nothing, which lowers the quality threshold considerably.
2. **Immediate timestamp capture reduces friction.** The most consistently praised feature was the “Capture current timestamp” button in the manual authoring form. In WordPress, setting a specific timestamp required noting the time from the player, navigating to a different screen, locating the timestamp field, and typing the value manually. On the new platform, clicking once while the video is paused captures the exact current position. Several participants remarked that they no longer worried about getting the timestamp wrong—because they no longer had to remember it at all.
3. **Progressive streaming reduces anxiety.** All ten participants preferred the SSE progressive streaming interface, where suggestions appeared one by one as they were generated, over a hypothetical batch interface where they would wait 15 seconds for all results at once. The most common phrasing was that seeing the system “doing something” meant they did not worry it had crashed. This aligns directly with

Nielsen’s “Visibility of System Status” heuristic [22]. The technical decision to use SSE rather than a simple HTTP POST with a long timeout had a measurable psychological effect on user confidence.

4. **AI quality matters less than AI availability.** Participants were more tolerant of imperfect AI suggestions than expected. Several edited the question text or changed the answer before accepting, without frustration. The consistent feedback was that having a draft to react to was valuable regardless of quality, because the hardest part of the task is deciding *what* to ask at a given point in the video, not how to phrase it. The AI answers this harder question; the instructor corrects the easier one.
5. **Suggestion editing was common but lightweight.** Exit interview data showed that participants accepted approximately 60–70% of AI-generated suggestions without modification, edited around 20–30%, and rejected the remainder outright. The edits were predominantly phrasing adjustments rather than content corrections, which is consistent with the expert quality ratings in Chapter 4 (93% relevance, 100% factual accuracy). The most common rejection reason was question redundancy — two suggestions covering the same concept from adjacent transcript segments — rather than factual error.

Final Synthesis.

The evaluation results point to two complementary sources of the platform’s advantage.

The AI generation feature is valuable, but it works best as a drafting aid under instructor supervision rather than an autonomous authoring system. The qualitative data makes this clear: participants who treated the AI suggestions as a starting point for editing performed better and reported higher satisfaction than the two participants who initially tried to use every suggestion without reviewing them.

The more fundamental contribution is the removal of workflow friction. Bringing video playback, timestamp capture, question authoring, and AI review into a single unbroken screen eliminated the context-switching that made WordPress so slow and error-prone. The total number of required interface steps to create five interactions and export dropped from approximately 47 (WordPress) to 8 (AI-ActivEdu). No individual change—not the AI, not the Zustand state management, not the one-click export—fully accounts for the 13-minute reduction in median task time. It is the combination. And the combination only works because all the pieces are in one place.

The platform’s outcome is consistent with Technology Acceptance Model predictions [18]: when perceived ease of use rises substantially, adoption of the underlying tool should follow. Whether that adoption materialises in practice at VNU-HCM is an empirical question that a longitudinal study would need to answer.

5.4 Threats to Validity

Three categories of threat affect how broadly these findings can be interpreted.

Internal validity. The study used a within-subjects design — each participant used both platforms — which controls for individual skill differences but introduces order effects. Platform order was counterbalanced by alternating the starting platform across participants: odd-numbered participants began with WordPress, even-numbered participants began with the new platform. However, learning transfer cannot be ruled out entirely: experience gained on one platform may have influenced performance on the second regardless of order. In addition, task difficulty was not formally verified to be equivalent across both sessions: participants chose their own interaction types and timestamps, so the cognitive demand of each individual session varied. Although the choice of interaction types was unconstrained on both platforms, there is no guarantee that participants selected equally challenging combinations in each session.

Construct validity. SUS measures perceived usability, not actual pedagogical effectiveness. The study confirms that instructors can author H5P content faster and with higher self-reported satisfaction; it does not confirm that the resulting interactive videos improve student learning outcomes. Similarly, task completion rate measures whether participants finished a prescribed lab task, not whether they could independently author content for their own courses. The AI suggestion quality evaluation used a single expert rater (the project supervisor) who blind-rated 30 randomly selected suggestions. Inter-rater reliability was not assessed, as only one rater was available. A multi-rater study using a standardised protocol such as Cohen’s kappa [?] would provide stronger evidence for the quality ratings reported in Table 4.2.

External validity. The ten participants were senior IT students from a single institution acting as educator proxies, not actual university faculty. This sample lacks diversity in academic background, discipline, and digital literacy level. Instructors from non-technical disciplines—humanities, social sciences, arts—may face different challenges, particularly when interpreting Bloom’s taxonomy labels or recovering from AI generation errors. Given the small sample size and non-normal distribution of time-on-task data, the Wilcoxon signed-rank test would be the appropriate non-parametric test for formally comparing the two platforms; however, the sample of ten participants does not provide sufficient statistical power to draw generalisable conclusions. The sample size satisfies Nielsen’s threshold for formative usability evaluation [22] but results should be treated as indicative rather than generalisable pending a larger, more diverse study with actual faculty participants.

5.5 Research Question Answers

RQ1: What are the specific usability barriers in current H5P authoring workflows?

The primary barriers identified through interviews, observation, and think-aloud protocols are: multi-screen navigation fragmentation (47 interface interactions across 6 screens); the cognitive burden of managing CMS concepts irrelevant to the teaching task; the absence of real-time preview during form entry; and no AI scaffolding for question generation. These four barriers map directly to the four problem dimensions defined in Chapter 1.

RQ2: How can user-centred design reduce extraneous cognitive load?

Co-locating the video player, timestamp capture, question editor, and AI suggestion panel on a single viewport eliminates the split-attention effect identified by Chandler and Sweller [12]. The interaction count dropped from 47 to 8 steps. This structural change drove the 13-minute reduction in median task time more than any individual feature. Heuristic evaluation during the Figma prototype stage caught and resolved three specific usability problems before formal testing, confirming the value of early expert review [27].

RQ3: What full-stack architecture best supports AI-assisted H5P authoring?

A multi-provider LLM architecture with Zod-based structured output validation works well for this use case. The “output automater” prompting pattern [30] forces the LLM to conform to strict JSON schemas, preventing hallucination of unknown field names or malformed structures. SSE streaming is essential for managing perceived latency: a 1.2-second Time-to-First-Token with an 8.4-second total generation time feels far more acceptable than a 15-second blank wait (confirmed by all ten participants). Decoupling H5P from a CMS using the headless Lumieducation library [15] enables a clean three-tier REST architecture scalable to institutional deployment [33].

RQ4: To what extent does the platform outperform the baseline?

Task completion rose 65 percentage points (30% to 95%), time-on-task fell 72% (18 minutes to 5 minutes), and the mean SUS score moved from 38.0 (“Poor”) to 82.5 (“Excellent”) on Brooke’s scale [28]. These are large effects across all three operationalisations of usability, efficiency, and productivity. The magnitude suggests the differences are practically significant and not attributable to measurement error, though the sample size precludes formal statistical inference.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The aims of this thesis were to reduce the complexity of the H5P interactive video authoring workflow by designing a user-centred interface, and to make AI-assisted question generation viable for use in the production of interactive videos at a Vietnamese university. Both aims were achieved in a single web platform designed from scratch and tested against the dominant WordPress H5P workflow.

The core technical result is an architecture that integrates a React 18 + TypeScript SPA [34, 35], a stateless Express.js REST API [41], a SQLite data tier via Sequelize ORM [42, 43], and the Lumieducation headless H5P server library [15]. An AI pipeline extracts timestamped transcripts from video sources (YouTube captions, uploaded WebVTT files, or local Whisper transcription [32]), passes them to a large language model (`llama-3.3-70b-versatile` via Groq, with a local Ollama fallback), validates the structured output with Zod schemas, and streams results to the browser via Server-Sent Events. Each suggestion carries a Bloom’s taxonomy classification so instructors can gauge cognitive depth at a glance [10].

The platform was evaluated against the WordPress H5P workflow in a comparative usability study with ten participants, each manually creating five randomly chosen interactions on the same video. Task completion rose from 30% to 95%; median task time fell from 18 minutes to 5 minutes; and the mean SUS score moved from 38.0 to 82.5. These improvements hold across every metric, which makes it difficult to attribute them to any single design choice. Putting all authoring tools on one screen, removing the need to switch between pages, and adding AI suggestion review together removed the barriers that made the WordPress workflow so slow.

Revisiting the Research Questions.

Full answers to each research question are developed in Chapter 5, Section 5.5. The key conclusions are summarised here.

RQ1 identified four principal usability barriers in the WordPress H5P workflow: multi-screen navigation fragmentation, CMS cognitive overhead irrelevant to the teaching task, absence of real-time preview, and no AI question scaffolding. All four were addressed in the new platform. RQ2 showed that co-locating all authoring components on a single viewport — eliminating the split-attention effect [12] — reduced the interaction count from 47 to 8 steps and drove most of the 13-minute task-time reduction. RQ3 established that a multi-provider LLM pipeline with Zod-based output validation and SSE streaming balances generation quality, perceived latency, and structural reliability. RQ4 found that task completion rose from 30% to 95%, median time-on-task fell from 18 minutes to 5 minutes, and the mean SUS score moved from 38.0 (“Poor”) to 82.5 (“Excellent”) on Brooke’s scale [28]. These outcomes are consistent with Technology Acceptance Model predictions [18] that reductions in perceived effort raise adoption likelihood, though confirming this causal link requires a longitudinal study.

Summary of Contributions.

Five contributions come out of this work:

1. **A production-ready authoring platform:** An open-source, CMS-free H5P authoring environment designed for institutional deployment.
2. **A functional AI-to-H5P pipeline:** The first documented integration of a production LLM pipeline directly generating complex, schema-compliant H5P content types guided by Bloom’s taxonomy.
3. **Structured output prompt engineering:** Reusable prompt matrices and Zod validation strategies that address the problem of forcing LLMs to produce valid nested instructional JSON.
4. **Empirical evidence for UCD in EdTech:** Quantitative confirmation that a purpose-built user-centred interface substantially outperforms a general-purpose CMS workflow for this specific task.
5. **Alignment with national digital education goals:** A locally deployable, open-source tool that supports VNU-HCM’s digital transformation strategy without commercial licensing costs [3].

Limitations of the Study.

Several limitations constrain how broadly these findings can be interpreted. The ten-participant sample satisfies Nielsen’s guidance for identifying usability problems [22], but lacks the statistical power for parametric hypothesis testing. The participants were technically literate IT students, not actual faculty members. University professors may

behave differently, particularly in how they respond to AI suggestions and recover from errors.

The AI quality evaluation used English-language transcripts. The platform technically supports Vietnamese prompts and bilingual UI strings, but the pedagogical quality of Vietnamese-language output has not been formally evaluated with human raters. This is a significant gap given the primary target institution. The Whisper transcription accuracy on Vietnamese academic speech with heavy domain-specific vocabulary is also an open question.

Finally, this evaluation measured short-term usability. Long-term adoption rates, frequency of use over a full academic semester, and the actual impact of the generated interactive videos on student assessment scores are all outside the scope of this initial study.

6.2 Future work

The modular architecture opens several natural directions for extension.

Longitudinal Faculty Evaluation Study. A semester-long study with actual university faculty across diverse disciplines at VNU-HCM would provide the most ecologically valid evidence for the platform’s impact. Measuring organic adoption rates, content production volume, and student performance data over a full academic term would answer questions this initial study could not.

AI Provider Upgrade for Greater Concurrency. The Groq free-tier API limits simultaneous AI generation to roughly one or two concurrent requests before queuing begins. Upgrading to a paid Groq tier, or switching the primary provider to a higher-capacity API such as Anthropic Claude or OpenAI GPT-4o, would allow the platform to serve a full classroom of instructors generating content simultaneously without rate-limit delays. A longer-term option is deploying a dedicated GPU inference server running an open-weight model such as Llama-3.3-70B, which eliminates per-request API costs and rate limits entirely for institutional deployments.

Additional H5P Question Types. The current platform supports six interaction types. Expanding the AI pipeline to cover additional types—Branching Scenario (for conditional learning paths), Course Presentation (slide-based mini-lectures with embedded questions), and Hotspot (click-on-image identification tasks)—would substantially broaden the range of instructional designs possible within the platform. Each new type requires adding a corresponding entry to the pattern matrix and a Zod validation schema for its JSON structure.

Vietnamese Language Validation. A dedicated evaluation study using real Vietnamese lecture transcripts and bilingual instructional designers as raters is needed to formally assess the pedagogical quality of Vietnamese-language AI output. Fine-tuning the local Whisper model on VNU-HCM’s academic terminology would also improve transcription accuracy for domain-specific content.

Deep Learning Analytics and xAPI Integration. The current platform exports standard .h5p packages but does not capture learner interaction data. Implementing xAPI tracking that reports events (video replays, question failures, time per interaction) to a Learning Record Store would unlock learning analytics, allowing instructors to see which specific concepts students are struggling with [36].

Collaborative Real-Time Authoring. Adding real-time collaborative editing would enable team-based content production between senior professors and teaching assistants. The backend already uses Express, so integrating WebSockets via `Socket.io` to broadcast state changes across connected clients is architecturally straightforward.

Local LLM Fine-Tuning. The current pipeline relies on general-purpose LLMs constrained by complex prompt engineering. Fine-tuning a smaller open-weight model (7B or 8B Llama variant) on a curated dataset of validated H5P question-transcript pairs from the VNU-HCM MOOC platform would improve suggestion quality and reduce inference latency for the local Ollama fallback path.

Asynchronous Media Processing. FFmpeg thumbnail generation and Whisper audio processing currently run synchronously in the main Express request handler, causing latency spikes on video upload. Moving these to an asynchronous job queue using Redis and BullMQ would decouple heavy computation from the API response cycle and improve scalability under institutional load.

WCAG 2.1 AA Accessibility Audit. The current frontend uses basic accessibility primitives (keyboard focus trapping, semantic HTML, ARIA labels), but the interactive timeline and dynamic AI review panels have not been formally audited for WCAG 2.1 AA compliance. Full accessibility compliance is an ethical and legal prerequisite before the platform can be officially adopted as an institutional standard.

REFERENCES

- [1] Helmet Contributors. Helmet: Help secure Express apps with HTTP headers. <https://helmetjs.github.io>, 2023. Accessed: 2025.
- [2] OWASP Foundation. OWASP top ten 2021. <https://owasp.org/www-project-top-ten/>, 2021. Accessed: 2025.
- [3] Vietnamese Government. National education development strategy 2021–2030, decision no. 1373/qđ-ttg. <https://thuvienphapluat.vn/van-ban/Giao-duc/Quyet-dinh-1373-QĐ-TTg-2021-phat-trien-giao-duc-2021-2030-488936.aspx>, 2021. Accessed: 2025.
- [4] Vietnam National University Ho Chi Minh City. VNU-HCM MOOC Platform. <https://courses.vnuhcm.edu.vn>, 2023. Accessed: 2025.
- [5] Kennedy Hadullo, Robert Oboko, and Elijah Omwenga. Issues affecting asynchronous e-learning quality in developing countries university settings. *International Journal of Education and Development using Information and Communication Technology*, 13(1):101–114, 2017.
- [6] Dongsong Zhang, Lina Zhou, Robert O. Briggs, and Jay F. Nunamaker. Instructional video in e-learning: Assessing the impact of interactive video on learning effectiveness. *Information & Management*, 43(1):15–27, 2006.
- [7] Scott Freeman, Sarah L. Eddy, Miles McDonough, Michelle K. Smith, Nnadozie Okoroafor, Hannah Jordt, and Mary Pat Wenderoth. Active learning increases student performance in science, engineering, and mathematics. *Proceedings of the National Academy of Sciences*, 111(23):8410–8415, 2014.
- [8] Martin Merkt, Sonja Weigand, Anja Heier, and Stephan Schwan. Learning with videos vs. learning with print: The role of interactive features. *Learning and Instruction*, 21(6):687–704, 2011.
- [9] Ömer Faruk Vural. The impact of a question-embedded video-based learning tool on e-learning. *Educational Sciences: Theory & Practice*, 13(2):1315–1323, 2013.

- [10] Lorin W. Anderson and David R. Krathwohl. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.
- [11] John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.
- [12] Paul Chandler and John Sweller. Cognitive load theory and the format of instruction. *Cognition and Instruction*, 8(4):293–332, 1991.
- [13] Fred Paas, Alexander Renkl, and John Sweller. Cognitive load theory and instructional design: Recent developments. *Educational Psychologist*, 38(1):1–4, 2003.
- [14] H5P Group. H5P: Open-source interactive content platform. <https://h5p.org>, 2023. Accessed: 2025.
- [15] Lumi Education GmbH. H5P Node.js Libraries – @lumieducation/h5p-server. <https://github.com/Lumieducation/H5P-Nodejs-library>, 2023. Accessed: 2025.
- [16] Moodle HQ. Moodle: Open-source learning management system. <https://moodle.org>, 2023. Accessed: 2025.
- [17] IMS Global Learning Consortium. Learning tools interoperability (LTI) core specification v1.3. <https://www.imsglobal.org/spec/lti/v1p3/>, 2019. Accessed: 2025.
- [18] Fred D. Davis. Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3):319–340, 1989.
- [19] Tom B. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- [20] Anthropic. Claude 3 model card. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf, 2024. Accessed: 2025.
- [21] Richard E. Mayer. *Multimedia Learning*. Cambridge University Press, 2nd edition, 2009.
- [22] Jakob Nielsen. *Usability Engineering*. Morgan Kaufmann Publishers, 1994.
- [23] D. R. Garrison and Norman D. Vaughan. *Blended Learning in Higher Education: Framework, Principles, and Guidelines*. Jossey-Bass, 2008.

- [24] International Organization for Standardization. ISO 9241-11:2018 ergonomics of human-system interaction – part 11: Usability: Definitions and concepts. <https://www.iso.org/standard/63500.html>, 2018.
- [25] Jesse James Garrett. *The Elements of User Experience: User-Centered Design for the Web and Beyond*. New Riders, 2nd edition, 2010.
- [26] Donald A. Norman. *The Design of Everyday Things*. Basic Books, 1988.
- [27] Rolf Molich and Jakob Nielsen. Improving a human-computer dialogue. *Communications of the ACM*, 33(3):338–348, 1990.
- [28] John Brooke. SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, editors, *Usability Evaluation in Industry*, pages 189–194. Taylor and Francis, 1996.
- [29] Gwo-Jen Hwang, Hui Xie, Ben Wei Wah, and Dragan Gašević. Vision, challenges, roles and research issues of artificial intelligence in education. *Computers and Education: Artificial Intelligence*, 1:100001, 2020.
- [30] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. A prompt pattern catalog to enhance prompt engineering with ChatGPT. *arXiv preprint arXiv:2302.11382*, 2023.
- [31] Benjamin S. Bloom, Max D. Engelhart, Edward J. Furst, Walker H. Hill, and David R. Krathwohl. *Taxonomy of Educational Objectives: The Classification of Educational Goals*. Longmans, Green, 1956.
- [32] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [33] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 3rd edition, 2012.
- [34] Meta Open Source. React 18: A JavaScript library for building user interfaces. <https://react.dev>, 2023. Accessed: 2025.
- [35] Microsoft. TypeScript: JavaScript with syntax for types. <https://www.typescriptlang.org>, 2023. Accessed: 2025.
- [36] George Siemens and Phil Long. Penetrating the fog: Analytics in learning and education. *EDUCAUSE Review*, 46(5):30–32, 2011.
- [37] Ian Sommerville. *Software Engineering*. Pearson, 10th edition, 2016.

- [38] Kent Beck et al. Manifesto for agile software development. <https://agilemanifesto.org>, 2001.
- [39] Vite Team. Vite: Next generation frontend tooling. <https://vitejs.dev>, 2023. Accessed: 2025.
- [40] OpenJS Foundation. Node.js: JavaScript runtime built on Chrome’s V8 engine. <https://nodejs.org>, 2023. Accessed: 2025.
- [41] OpenJS Foundation. Express: Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com>, 2023. Accessed: 2025.
- [42] SQLite Consortium. SQLite: A self-contained, high-reliability SQL database engine. <https://www.sqlite.org>, 2023. Accessed: 2025.
- [43] Sequelize Team. Sequelize v6: Node.js ORM for SQL databases. <https://sequelize.org>, 2023. Accessed: 2025.
- [44] FFmpeg Developers. FFmpeg: A complete, cross-platform solution to record, convert, and stream audio and video. <https://ffmpeg.org>, 2023. Accessed: 2025.
- [45] Michael Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, IETF, 2015.

APPENDIX

A.1 SYSTEM SETUP AND API REFERENCE

A.1.1 Runtime Environment

Table 1 lists the software versions used during development and testing.

Table 1: Key runtime components (representative versions)

Component	Notes
Node.js 18 LTS	Backend runtime; required by H5P server package.
React 18 + Vite	Frontend runtime and build tool (TypeScript used throughout).
SQLite (sqlite3)	Lightweight relational store used in development; Sequelize ORM.
AI tooling	Optional: OpenAI/Anthropic APIs, Ollama local inference, faster-whisper for transcription.
Utilities	Nodemailer (email), Helmet (security), CORS and rate-limiting middleware.

A.1.2 Starting the Platform

1. **Install backend dependencies:**

```
1 cd backend && npm install
```

2. **Configure environment:** Copy `backend/.env.example` to `backend/.env` and set `JWT_SECRET` and `GROQ_API_KEY` (required for AI generation).

3. **Start the backend** (runs on port 5001 by default):

```
1 cd backend && npm run dev
```

4. **Install frontend dependencies:**

```
1 cd frontend && npm install
```

5. **Start the frontend** (runs on port 3002 by default):

```
1 cd frontend && npm run dev
```

6. Open the application at <http://localhost:3002> in a modern web browser.

A.1.3 Key API Endpoints Reference

Table 2: Key API endpoints

Method	Path	Description
POST	/api/auth/register	Create a new user account.
POST	/api/auth/login	Authenticate and receive JWT.
GET	/api/videos	List authenticated user's videos.
POST	/api/videos	Upload a new video file.
POST	/api/videos/youtube	Import a YouTube video by URL.
GET	/api/videos/:id	Get a single video's metadata.
DELETE	/api/videos/:id	Soft-delete a video.
GET	/api/h5p/:videoId/content	Get H5P interactions for a video.
POST	/api/h5p/:videoId/content	Add an H5P interaction manually.
POST	/api/h5p/video/:videoId/export	Export H5P package (binary download).
GET	/api/ai/status	Check AI provider availability.
POST	/api/ai/analyze	Analyse transcript (non-streaming).
POST	/api/ai/analyze-stream	Analyse transcript (SSE streaming).
POST	/api/ai/inject	Inject accepted suggestions into H5P.
POST	/api/transcript/youtube	Extract captions from YouTube video.
GET	/api/admin/users	List all users (admin only).
PATCH	/api/admin/users/:id	Update user role or active status (admin only).
POST	/api/admin/users	Create a new account (admin only).
GET	/api/admin/audit-log	Retrieve admin audit log (admin only).
GET	/api/admin/settings	Get all system settings (admin only).
PUT	/api/admin/settings/:key	Update a system setting (admin only).
PUT	/api/users/profile	Update authenticated user's profile.
POST	/api/auth/verify-email	Submit 6-digit email verification code.
POST	/api/auth/forgot-password	Request password reset email.
POST	/api/auth/reset-password	Submit new password with reset token.
GET	/api/lti/launch	LTI 1.3 launch endpoint for LMS integration.